Project Week 07-11 copy

March 27, 2025

0.0.1 2 Week 7 [Feb 10 - Feb 16]: Collaborative Filtering Recommender System

1. Based on the frequency of the most rated items computed in Week 6, implement the TopPop recommender system, which always recommends the same top-k items sorted decreasingly by the number of "high" ratings (e.g., 3) in the training split, train.tsv.

[115]:
```
import pandas as pd

class TopPop:
    def __init__(self, k=10):
        self.k = k
        self.top_items = None

    def fit(self, file_path):
        # Read the training data (no header assumed)
        df = pd.read_csv(file_path, sep='\t')
        # Keep only the high ratings (e.g., >= 3)
        df_high = df[df['rating'] >= 3]
        # Compute the frequency of high ratings per item and sort in decreasing␣
 ↪order
        counts = df_high['item_id'].value_counts()
        self.top_items = list(counts.index)

    def recommend(self, user_id):
        # Return the top-k items (this model is user-agnostic)
        return self.top_items[:self.k]

top_pop = TopPop(k=10)
top_pop.fit('Data/clean_train.tsv')
```

2. Choose at least one neighborhood-based model and one latent factor model that uses the observed user-item ratings in the training set to predict the unobserved ratings. Report your choice of models.

[116]:
```
from surprise import Dataset, Reader, KNNWithMeans, SVD
import pandas as pd

# Load your training data from the file (in 'Data/clean_train.tsv')
df = pd.read_csv('Data/clean_train.tsv', sep='\t')
```

```
reader = Reader(rating_scale=(1, 5))

# Prepare the dataset using the surprise framework
data = Dataset.load_from_df(df[['user_id', 'item_id', 'rating']], reader)
trainset = data.build_full_trainset()
```

3. Use 5-fold cross-validation on the training set to tune the hyperparameters of the chosen models (similarity measure and number of neighbors for the neighborhood-based model; number of latent factors and number of epochs for the latent factor model).

[117]:
```
from surprise.model_selection import GridSearchCV
import random
import numpy as np

# Set the random seed for reproducibility
random.seed(42)
np.random.seed(42)

# Define parameter grid for KNNWithMeans (neighborhood-based model)
param_grid_knn = {
    'k': [5, 10, 20, 30, 40, 50, 70, 100],
    'verbose': [False],
    'sim_options': {
        'name': ['cosine', 'msd'],
        'user_based': [True]  # We use user-based collaborative filtering
    },
    'random_state': [42]
}

# Define parameter grid for SVD (latent factor model)
param_grid_svd = {
    'n_factors': [3, 5, 10, 20, 50, 100],
    'n_epochs': [5, 10, 20, 50, 100],
    'random_state': [42]
}

# Perform 5-fold cross-validation for KNNWithMeans
grid_knn = GridSearchCV(KNNWithMeans, param_grid_knn, measures=['rmse', 'mae'],␣
 ↪cv=5)
grid_knn.fit(data)

# Perform 5-fold cross-validation for SVD
grid_svd = GridSearchCV(SVD, param_grid_svd, measures=['rmse', 'mae'], cv=5)
grid_svd.fit(data)
```

4. Choose an evaluation measure that is suitable for this task and justify your motivation in using it. Report the optimal hyperparameters together with the scores of your chosen measure, averaged over the 5 folds.

[118]:
```
# We choose RMSE (Root Mean Squared Error) as the evaluation measure

print("\nKNNWithMeans Best Hyperparameters (based on RMSE):")
print(grid_knn.best_params['rmse'])
print("Average RMSE for KNNWithMeans:", grid_knn.best_score['rmse'])

print("\nSVD Best Hyperparameters (based on RMSE):")
print(grid_svd.best_params['rmse'])
print("Average RMSE for SVD:", grid_svd.best_score['rmse'])
```

```
KNNWithMeans Best Hyperparameters (based on RMSE):
{'k': 70, 'verbose': False, 'sim_options': {'name': 'cosine', 'user_based':
True}, 'random_state': 42}
Average RMSE for KNNWithMeans: 0.9100087207501553

SVD Best Hyperparameters (based on RMSE):
{'n_factors': 5, 'n_epochs': 20, 'random_state': 42}
Average RMSE for SVD: 0.8440025192355
```

5. Run the models with the optimal hyperparameters to the whole training set.

[119]:
```
# Re-train the models on the full training set using the optimal␣
 ↪hyperparameters from the grid searches

# For KNNWithMeans
knn_best = grid_knn.best_estimator['rmse']
knn_best.fit(trainset)

# For SVD
svd_best = grid_svd.best_estimator['rmse']
svd_best.fit(trainset)

print("KNNWithMeans model trained with optimal hyperparameters.")
print("SVD model trained with optimal hyperparameters.")
```

```
KNNWithMeans model trained with optimal hyperparameters.
SVD model trained with optimal hyperparameters.
```

6. Use the final models to rank the non-rated items for each user. This ranking will be used for the evaluation part next week.

[120]:
```
from collections import defaultdict

# Build the anti-testset (all user-item pairs not in the training set)
anti_testset = trainset.build_anti_testset()

# Get predictions on the anti-testset using the KNNWithMeans model
predictions = knn_best.test(anti_testset)
```

```python
# Group predictions by user
user_recs = defaultdict(list)
for pred in predictions:
    user_recs[pred.uid].append((pred.iid, pred.est))

# For each user, sort the recommendations in descending order of predicted␣
 ↪rating
for user in user_recs:
    user_recs[user] = [iid for iid, _ in sorted(user_recs[user], key=lambda x:␣
 ↪x[1], reverse=True)]

# Display sample recommendations for a few users
print("Sample recommendations using KNNWithMeans:")
for user, recs in list(user_recs.items())[:5]:
    print(f"User {user}: {recs[:10]}")

# Get predictions on the anti-testset using the SVD model
predictions = svd_best.test(anti_testset)

# Group predictions by user
user_recs = defaultdict(list)
for pred in predictions:
    user_recs[pred.uid].append((pred.iid, pred.est))

# For each user, sort the recommendations in descending order of predicted␣
 ↪rating
for user in user_recs:
    user_recs[user] = [iid for iid, _ in sorted(user_recs[user], key=lambda x:␣
 ↪x[1], reverse=True)]

# Display sample recommendations for a few users
print("Sample recommendations using SVD:")
for user, recs in list(user_recs.items())[:5]:
    print(f"User {user}: {recs[:10]}")
```

```
Sample recommendations using KNNWithMeans:
User AGTVZ7ZSDMTEDLMJGZRC7RFJNDWQ: ['B0C67HCGBR', 'B0BPJ4Q6FJ', 'B0B8M5FJB6',
'B00H35YIJE', 'B079TLFL33', 'B000T9L7W2', 'B007MY5BDI', 'B00H4PEMM6',
'B09YDBKT7M', 'B0002D0Q2W']
User AG44UYFDZHI6FRBQSLDWOGIEOY4A: ['B0C67HCGBR', 'B0B8M5FJB6', 'B000T9L7W2',
'B007MY5BDI', 'B09YDBKT7M', 'B079P9LDHN', 'B0BPKTYTPQ', 'B001W99HE8',
'B000J5UEGQ', 'B01DBS2U9G']
User AE7P3HIBI3UDLCJLUPUZQWYVPEWA: ['B005M0MUQK', 'B0BRS6V8G4', 'B078L11275',
'B09R6KV6QX', 'B07YK57N2M', 'B00RX5HQS4', 'B0BQ4HSKC9', 'B086QM1F75',
'B07R3S93K8', 'B09WF82F1V']
User AE5M7M2VYMM3WHJIBLCHHEU7UOXQ: ['B06XB3FQKB', 'B0BPJ4Q6FJ', 'B0B8M5FJB6',
```

```
'B079TLFL33', 'B007MY5BDI', 'B00H4PEMM6', 'B09YDBKT7M', 'B0BXT384GR',
'B079P9LDHN', 'B0BPKH4HB2']
User AESDNHQRRZEWTZF7A7TFFFTSNY5Q: ['B0BGQZNQ53', 'B07L5B64RG', 'B07Y94MSG9',
'B0CB98SMQR', 'B079NS31NK', 'B00CIHB8FY', 'B000SHQ1QC', 'B0BQ4HSKC9',
'B00IZA1GI2', 'B000U0DU34']
Sample recommendations using SVD:
User AGTVZ7ZSDMTEDLMJGZRC7RFJNDWQ: ['B0BPJ4Q6FJ', 'B00H35YIJE', 'B079TLFL33',
'B000T9L7W2', 'B007MY5BDI', 'B09YDBKT7M', 'B079P9LDHN', 'B0BPKH4HB2',
'B07JCW1KGG', 'B001W99HE8']
User AG44UYFDZHI6FRBQSLDWOGIEOY4A: ['B000T9L7W2', 'B007MY5BDI', 'B09YDBKT7M',
'B079P9LDHN', 'B001W99HE8', 'B01DBS2U9G', 'B07CBT4SYD', 'B09M7DPHCS',
'B08DM9NFLH', 'B0BTC9YJ2W']
User AE7P3HIBI3UDLCJLUPUZQWYVPEWA: ['B0BT2ZRCY2', 'B07DK59QNR', 'B000U0DU34',
'B09G5KLKX2', 'B01DE4D4LU', 'B0B8F6LD9F', 'B0BT9R8MMV', 'B07N5MJDBF',
'B07S19XSPV', 'B0CB98SMQR']
User AE5M7M2VYMM3WHJIBLCHHEU7UOXQ: ['B07N5MJDBF', 'B0BT9R8MMV', 'B0BT2W3TTM',
'B07N2HQ1T7', 'B000U0DU34', 'B09M7BF885', 'B09G5KLKX2', 'B0BRS6V8G4',
'B01DE4D4LU', 'B07DK59QNR']
User AESDNHQRRZEWTZF7A7TFFFTSNY5Q: ['B000U0DU34', 'B0BT2ZRCY2', 'B0BT9R8MMV',
'B0B8F6LD9F', 'B01DE4D4LU', 'B09G5KLKX2', 'B07C9YCY5J', 'B0BQ4HSKC9',
'B07N5MJDBF', 'B0CB98SMQR']
```

0.0.2 3 Week 8 [Feb 17 - Feb 23]: Evaluation of Recommender Systems

In this session, we will discuss how to evaluate a recommender system. Specifically, let us evaluate all your recommender models from Week 7 on the preprocessed test data split studied in Week 6. You need to: - Measure the error of the system's predicted likelihood of rating for the items (Root Mean Square Error, RMSE). - Discuss the limitations of this metric.

[121]:
```python
from surprise import accuracy

# Load the preprocessed test data (assuming same structure as train data: user_id, item_id, rating)
test_df = pd.read_csv('Data/clean_test.tsv', sep='\t')

# Convert the test DataFrame to a list of (user, item, rating) tuples
testset = [tuple(x) for x in test_df[['user_id', 'item_id', 'rating']].values]

# Evaluate SVD model on the test set
svd_predictions = svd_best.test(testset)
print("SVD RMSE on test set:")
accuracy.rmse(svd_predictions, verbose=True)

# Evaluate KNNWithMeans model on the test set
knn_predictions = knn_best.test(testset)
print("KNNWithMeans RMSE on test set:")
accuracy.rmse(knn_predictions, verbose=True)
```

```
SVD RMSE on test set:
RMSE: 0.9795
KNNWithMeans RMSE on test set:
RMSE: 1.0619
```

[121]:
```
1.0618532415444242
```

Now, we are interested not in whether the system properly predicts the rating of these items, but rather whether the system gives the best recommendations for each user. To evaluate this, generate the top-k (with k = 10) recommendation for each test user. Based on the top-k recommendation list generated for each user, and using the test data split5, compute: - Hit rate, averaged across users. - Precision@k, averaged across users - Mean Average Precision (MAP@k) - Mean Reciprocal Rank (MRR@k) - Coverage

Discuss the advantages and disadvantages of these metrics. Compare the two types of CF recommender systems and the TopPop system that we have defined so far. Which one works best? Why? What are the advantages and limitations of each approach?

[122]:
```
# Prepare ground truth for each test user (using binary labels)
# Relevant (1) if rating >= 4; not relevant (0) otherwise.
ground_truth = defaultdict(set)
for _, row in test_df.iterrows():
    if row['rating'] >= 4:
        ground_truth[row['user_id']].add(row['item_id'])

# Function to generate top-k recommendations using a given model.
# For personalized models we use the anti-testset from the training set.
def get_top_k(model, trainset, k=10):
    # Build the anti-testset (all unseen items for every user in the trainset)
    anti_testset = trainset.build_anti_testset()
    predictions = model.test(anti_testset)

    recommendations = defaultdict(list)
    for pred in predictions:
        recommendations[pred.uid].append((pred.iid, pred.est))

    # For each user, sort predicted items in descending order and select top-k.
    for user in recommendations:
        recommendations[user] = [iid for iid, _ in␣
 ↪sorted(recommendations[user], key=lambda x: x[1], reverse=True)[:k]]
    return recommendations

# For TopPop (non-personalized), the top-k recommendations are the same for␣
 ↪every user.
def get_top_k_for_top_pop(top_pop, test_users, k=10):
    recs = {}
    top_items = top_pop.top_items[:k]
    for user in test_users:
```

```
        recs[user] = top_items
    return recs

# Generate recommendations for each model.
# Retrieve the list of test users from ground_truth.
test_users = list(ground_truth.keys())

svd_recs = get_top_k(svd_best, trainset, k=10)
knn_recs = get_top_k(knn_best, trainset, k=10)
top_pop_recs = get_top_k_for_top_pop(top_pop, test_users, k=10)

# Get the total catalog size (we use all items from the training split)
total_items = trainset.n_items

# Evaluation function for hit rate, precision@k, MAP@k, MRR@k and coverage.
def evaluate(recs, ground_truth, k=10, total_items=total_items):
    hit_rates = []
    precisions = []
    avg_precisions = []
    reciprocal_ranks = []
    rec_items = set()

    for user, true_items in ground_truth.items():
        recommended = recs.get(user, [])
        rec_items.update(recommended)
        # Hit rate: at least one relevant item recommended.
        hit = 1 if set(recommended) & true_items else 0
        hit_rates.append(hit)
        # Precision@k: fraction of recommended items that are relevant.
        precision = len(set(recommended) & true_items) / k
        precisions.append(precision)
        # Average Precision (AP@k)
        ap = 0
        hit_count = 0
        for i, item in enumerate(recommended):
            if item in true_items:
                hit_count += 1
                ap += hit_count / (i + 1)
        if len(true_items) > 0:
            ap /= len(true_items)
        avg_precisions.append(ap)
        # Reciprocal Rank (MRR@k): rank of the first relevant recommendation.
        rr = 0
        for i, item in enumerate(recommended):
            if item in true_items:
                rr = 1 / (i + 1)
                break
```

```
        reciprocal_ranks.append(rr)

    hit_rate = np.mean(hit_rates)
    precision_at_k = np.mean(precisions)
    map_at_k = np.mean(avg_precisions)
    mrr_at_k = np.mean(reciprocal_ranks)
    # Coverage: unique items recommended over catalog total.
    coverage = len(rec_items) / total_items

    return hit_rate, precision_at_k, map_at_k, mrr_at_k, coverage

# Evaluate each recommender.
svd_metrics = evaluate(svd_recs, ground_truth, k=10)
knn_metrics = evaluate(knn_recs, ground_truth, k=10)
top_pop_metrics = evaluate(top_pop_recs, ground_truth, k=10)

print("SVD:    Hit Rate, Precision@10, MAP@10, MRR@10, Coverage =", svd_metrics)
print("KNN:    Hit Rate, Precision@10, MAP@10, MRR@10, Coverage =", knn_metrics)
print("TopPop: Hit Rate, Precision@10, MAP@10, MRR@10, Coverage =",␣
 ↪top_pop_metrics)
```

```
SVD:    Hit Rate, Precision@10, MAP@10, MRR@10, Coverage = (0.0825242718446602,
0.00825242718446602, 0.00610439651075204, 0.02219140083217753,
0.11787819253438114)
KNN:    Hit Rate, Precision@10, MAP@10, MRR@10, Coverage = (0.10922330097087378,
0.011650485436893206, 0.00841981197641613, 0.03142144398212359,
0.587426326129666)
TopPop: Hit Rate, Precision@10, MAP@10, MRR@10, Coverage = (0.25728155339805825,
0.032524271844660196, 0.03386359713699836, 0.11868354137771613,
0.019646365422396856)
```

Analysis on the effect of the long-tail Sort the users by number of ratings in the train set and select the top 20% and last 20% of users.

[123]:
```
# Compute the number of ratings per user from the trainset
user_rating_counts = {trainset.to_raw_uid(uid): len(ratings) for uid, ratings␣
 ↪in trainset.ur.items()}

# Sort users by their number of ratings
sorted_users = sorted(user_rating_counts.items(), key=lambda x: x[1])
num_users = len(sorted_users)

# Determine the 20% count for top and bottom users
num_20_percent = int(num_users * 0.2)

# Select bottom 20% (users with fewest ratings) and top 20% (users with most␣
 ↪ratings)
```

```python
bottom_users = [user for user, count in sorted_users[:num_20_percent]]
top_users = [user for user, count in sorted_users[-num_20_percent:]]

print("Total number of users:", num_users)
print("Bottom 20% users and their review counts:")
for user in bottom_users:
    print(f"{user}: {user_rating_counts[user]}")
print("Top 20% users and their review counts:")
for user in top_users:
    print(f"{user}: {user_rating_counts[user]}")
```

```
Total number of users: 800
Bottom 20% users and their review counts:
AF2MGAQBAT3E4XQC7NNAQFYG4MIQ: 1
AGMYRILPTJDOK7WL5OYHYGRYVXMQ: 1
AEMZMINSQYQBEDD3TPDQZZFFWIQA: 1
AGDEYWGZKNGR36MRO7ZT3AB45AXA: 1
AHMUAUOHLITDKIKI4C3KVRWIBMAQ: 1
AFFOT7AORQAWKU6KKPDHR3FOA5AA: 1
AEXXX2MQRH3AHZMBZBP4RDNGRRWQ: 1
AFGJ42MV2H2CHEZNN3JPQ3RSYUHQ: 1
AESA6T5VF3ADXH5GYRX2RDMN6I6Q: 1
AEWWDVYR4YKLLOILS55VQISDPGYQ: 2
AHOUKQ3IRAEAMEHDB6U4AMB366NQ: 2
AGSP5XAQPQBUUXZHEZSC65FD7NOQ: 2
AH2RITMRQC3JL44WADB4GV23AFVQ: 2
AF63VW7YLF42A6STVGBLACD7RUWA: 2
AHMMZCKZMV7U5SPYVE6IG2YACR7Q: 2
AHMVWBJ4FNFOXJ6BXA5M7CEDHSNA: 2
AHRPODCFJ7UNWQKCLLBKN6BQIN7A: 2
AHWAYCEX3NCWJD32JTFVBFXKUHOQ: 2
AGHARSG3LRWDNOAM3HVDWKJJMS3Q: 2
AE2NWSTL7JJJWOCBKZCZF6KDQIZQ: 3
AGRGLPIRFOMTUIBIYJOUTTGZHABA: 3
AETS7HNSQIFRSMRCWQMHIWZ2IBOQ: 3
AGHJMMZCZKGM2XC7E32JDSAGRWHQ: 3
AHUIUCGSUO6B2KRQLQ2UWRYXTH5Q: 3
AF72XXUN4AGVL4K45BIXLKDIZRTQ: 3
AEYWZ24XFZCRP3PXNJB2TLXFX47A: 3
AEWQYNFYO55WCN2IVUVQET3OFS6A: 4
AE6ZIS54JBDY3UPZQGSXRJMPQ3OA: 4
AH44K3RPIQMVWG2G4EB5FPVIVUEQ: 4
AGYK2YYS35CDU2Z77ZQR7L3IAX4Q: 4
AFPBNLIVMV6PSXJFUYLMO3EYWBPA: 4
AGNQMZSUPRA3BEM44ITFIFXKGPXQ: 4
AH2APCDO2UOVZXSGM7VZR3XMWR3A: 4
AFVHSDXVXIX3ZM6JB32LZEAYTFTQ: 4
AEGPH37EQW2VFMQTLTMTD26JH6ZQ: 4
```

```
AEHDYSX54B3NYUIIEIHEGHIGJNMQ: 4
AF4VYLI2QY2N7S65KZHMIET7OGOQ: 4
AG6FYGUMXO4TSTPFWPFTNHJXC7WQ: 4
AGBQ3TX4QYWQQOW2DBADLGOJHQGA: 4
AGB5CRFACFQG6C3OB2TSE5JC5FHA: 5
AH6QSJEXBISHMQ3M3THWZAKMB72A: 5
AFPU3TXAOZ2VSTJAV5SDRF2XA6CA: 5
AFLMX52QL4FWD3PYMYQWWE36QHMA: 5
AEXLRZ34GTMM35NQRDZ4CDKKXLIA: 5
AF3FYICX6MT2J3E7RBI6WGC2DSVA: 5
AEOG5GP4YYOR6T7H4DTLTH5VXEVA: 5
AGAF5MHBOW2UYHNHSBXIXLKJXIKQ: 5
AGBVX6ERCZ5FBYQCLQNGTWJWKC5A: 5
AF2GPIP26H7E5EHDWC6P7RKZ3OUQ: 5
AFIMYZ3UMWWZKR2ZFTOQZ2BDGA6Q: 5
AHLRDVX6ROCCPNXTCZLY2KPVLCHA: 5
AFOCHHI44XKMG44TIDXRHY7R2H2Q: 5
AHICAEXRGOBNEKCK2KQYUI44SMVQ: 5
AE23LDQTB7L76AP6E6WPBFVYL5DA: 5
AE6JDIRVPTKC32Y7X3QQ3WAY2LIQ: 5
AGFSYLEFUQAJDTR55TENS6BZ56XQ: 5
AFJJWM45JXP7ONUSAYQWWEIVDLDQ: 5
AFDZU5HCDZTKTC6LL2KI5DQHQ2IA: 5
AEQJ4C6IRPWITBGZ3PVJJB6DLW3Q: 5
AHTOQ6BJJR67CXFLW7CGRTLA3A3A: 5
AEKTSZJEHS2AFYBWFIMASB43ZC5A: 5
AE5M7M2VYMM3WHJIBLCHHEU7UOXQ: 6
AHSEAF6NAXXWPPY3CSL2K24G5IOA: 6
AHL4BCH2O33HFN7KSBTIJUHCJYDA: 6
AHG4XUNNKMZUSMC7UI3YRS5YMD7A: 6
AFRUTEYFUJDAUASSQADZ3UQJHXAA: 6
AHETQRJ33CMSEUL4SDS6CJMHDU6A: 6
AHGODYYGTJFO5RXGWPSKFVXS25PQ: 6
AFHIW6ZJRTCQCT2BKFRDPY2RQWLA: 6
AENF7IROS6474V6TOKAONGVZHMVQ: 6
AG34YJOTCJWCEA6XGG3NWWHDSQIQ: 6
AG5CLB4M2P4GCGANQS4WS3JGHFOQ: 6
AGPQVGNBINLAEQ6IWHJBGSL6YXJA: 6
AGMIECBRRAHS3YUEBQNL7M766ZVQ: 6
AHAZ5ZPYUOSJP6GUP5AM2VMUFNTA: 6
AHEIWAU3PKILH7VPQZGXPBS5BKEQ: 6
AEUZQ2IHGQE75ZWT6SQ6NQTWKPKQ: 6
AGSVLMHAE6DKL34BW5AJANZ2HP3A: 6
AG4V6SXKSVYO5TTRJQRPLBSK25HQ: 6
AGRWNQCKOVXJEAHMQ5RWUMGVAP5Q: 6
AENFQD5EQBOPKB6TB7AOAD5QQCOA: 6
AG4VWLJ2X7PIBYDXYYUNFY5ONDRQ: 7
AG5S2NUNLTCXHX7H5UYE5LZXJK7A: 7
```

AEWLK6HFFQ44AJIC4YHECUU7L4QA: 7
AG6MB6ECQOKI5LMTUXYE7NCQYQQA: 7
AGOZJU464U2DQK5MI4YNE4CSCLCQ: 7
AEH4Y332KQ4FVFT5VTSILR2RGJ4A: 7
AGBWY2Q4KM2IH3WA45BSPOCKRJOA: 7
AEQHYWT6W3GRY36WBNLB7J6HWHPA: 7
AHLU6AWIBESPT72U26FRE2PVI5JQ: 7
AF2O4GMP7Z6ZV7WZGJM5MGT27GAA: 7
AG77NH35B3YCLZHVX7ZBWGCPKU7Q: 7
AHUZL5QVP2SC6GOXSKGNAYUZQZ6A: 7
AGAZUH6XYIKXLJHGMSB5WA5ZV2KQ: 7
AFCH3BNUJBRNZP53DEJTCOZ26BIA: 7
AEMSFZ4NLED647BP3ZCQEY3FNGKA: 7
AHVGICSVXM66E4SXX23PH7R3Q74Q: 7
AH7I57UVCR75DY4O5MDVVGWT2CJQ: 7
AE4LGWEQHGH3GNC3YEIWKBSZN34Q: 7
AHYQKMLIXHKFDHC7K2Y4JXM4YLOQ: 7
AFTF5Z4H3CXBKB2HET3J72K6MTUQ: 7
AEMISRDYQCHSNOXKV6TQ67X2TIOA: 7
AFR6WKFQBTKBG7HH7MOIHP7Y4IJQ: 7
AETRLZMHXJWHRPFLNPAT5KWVN2UQ: 7
AEOUOKA2NPLBOSG73KGJTVQIOK6Q: 7
AGSVU7HWV6AHG4QFFTVWDXNM3UJA: 7
AFN4MT6SZPZWDQ5D5AGJ6WP3E3YQ: 8
AEYU4OX6OE3CJ6BQH2KLSNXPKRPA: 8
AHMKNGTBHAIBOSCZMO3N7FDGPTIA: 8
AFKIK7F3PKMBCI2IKCQWH6ZY5DEA: 8
AGASE3XTUSEWWXX6RV5KD4NDXFAA: 8
AFIEQ7BNYXBQTLYIJRTBNKWQTKEQ: 8
AFDS5HV3F3ONORE7U7PMXMVMZAUA: 8
AFNKANPQUS5G3ENXFLAJ4CHWQGFQ: 8
AGY5HQSNTLDTDJRG533ZDFUAHMRQ: 8
AHC23DBZJ2VVCK6UGCM7NPFP7LQA: 8
AE76MPFI6EGHS7HMMTW3SXCLVVKA: 8
AER5I36DVGSOMSGX3ZQHYZGPESFA: 8
AEVUPHNKTJYUVF3LVHMYVZT4WLUQ: 8
AFUD46FQ5QP2KHEPMCNSWXMDZ5MA: 8
AFG7GOJ4XZ6MQA7EXJQ6BCK3Y2YQ: 8
AEJH4HUMFAN62AII4PTCGRGGAE7Q: 8
AFVSSJ5QSQBZBTAVR5XHF6UQQVBA: 8
AHPIKIUVO7ZBNW4K4OLRCDRWLQYA: 8
AF6NUCZR3MMBLJJCDNZS6PKZTOYQ: 8
AGTQ4C5UOLPC6RNAX573O2TU5S3Q: 8
AFRFYELVMFERMILMAIN3NY7YOPSA: 8
AESL6WMA3BO3ZXPTK6EAQJIOFTUQ: 8
AE5D5Q2I4H7CLGGXKPQPMXOG6JLA: 8
AGFIGINQSL73HWAKFPGSIYO3KU5Q: 8
AFJ66RVSHHQWD3MZRM6JNQB5NGXA: 8

```
AHCLEFILWZQNDMN4L3LC3DJLZNSQ: 8
AGWKQRZOH2KLRB3W72ZFRMOTZRXQ: 8
AGR2HGNGBX6TFVFHQFKXRPP4B22Q: 8
AHBV7TH4Z3WRRKXUIDAOPVUBIR4Q: 8
AHHJZCFUMYNQ4M2VAYCDX537ZBRQ: 8
AE4ZMURD4I73BSFCFQPGOWBOHXIA: 8
AG5QJWMXSGR6QK7CTCJKXRD7OSOQ: 8
AF5QL2TW5Z63NYQ7IKBJVHB5T2CQ: 8
AF3JMCAJ6IICLZJRLDYWCPSSEN2A: 8
AHFWBUOIMC5ESSI6SQWTVXDYQ55Q: 8
AHFEX4TZQJHG6BTYRZQED45HVT6A: 8
AHNMR3M7H4JDTEO5LXIKZC4DJUUA: 8
AEFPGSOOIM5GMCPX7FOCCH3UF6ZA: 8
AF3ZYMHHEW4XP4NHUIZXKVVSAXFQ: 8
AE7P3HIBI3UDLCJLUPUZQWYVPEWA: 9
AFVLYQWBCIHXVLH6X2FSXRMAHHXA: 9
AEHGPRL2ZLF6TY6JWBNFJNIDTE6Q: 9
AHSEUPHWABAKNUWPZJOLLGDM47FA: 9
AG2OYBLXDT4PNZZDRKYHEELJ75YQ: 9
AFXVMXIAMROWJSIZ2IQBTX2GF4LA: 9
AGLSNCKCAMBZTZKPFN3BY2ANTARQ: 9
AEYIGWWXTT5AD4ZONPSHMK53WR2Q: 9
AGSWTRJPZ7AVS2C33JGND5BQ6SMQ: 9
AHVFEBH4PD2V3XK2AZ44LFXTPZUA: 9
AGL72EW24IMRO4K5EJ6S3QGG365Q: 9
AG6CZXXQXXL3L7TE7XVHTW3DYKYA: 9
AFV2YRAITESEBOVQYURW6VGM24MA: 9
AH22LHGRVRIAJI32SBDMHNOKPPMQ: 9
AFHER2NWLJQHNQ4BRB4NZAG46VCQ: 9
Top 20% users and their review counts:
AGVJNAG7UKKZH2XTBDZ2HD5DOJCA: 15
AENBPEIFKODS2DZB7JNJMYQTZ46A: 15
AFZ4XEURSL7VMVPEKG5SHU3OAULQ: 15
AFTN36CZUYIXDWJYJD7O2LBXYOEA: 15
AEH6OLQHQQVSIU7BDLIHSMIBOGNQ: 15
AHI4QESUN4OC3NMOB6M5KCE6WBDQ: 15
AEEDYZE7SWKKVMVWYZD35VYCMAWQ: 15
AFLTPN2GUQICRVFNIGT7X6QUW5JQ: 15
AG6QMKE26TOI4SJCPJD2F3APWE7Q: 15
AFVN3Z7WL2CLXIBUTHYR6K6Y7YJA: 15
AHRDQWA7SWNXDYHRPEZ63D5Q7B5Q: 15
AHPMJNUYUX4QOZCKLN5R34BB5EQQ: 15
AEND7BD36I5IUIVPDZJAQXBLR5CA: 15
AG3TBHO3TDHQXXZR6IUHEZGAZ2MQ: 15
AENNORCDOTT2KFT5WUIK7PJ2EXOA: 15
AHOZBVGYQOXJE4NJRSPWY52ILAKQ: 16
AFGQTSA5VSAM326LO6ZKAIWIFK5A: 16
AEEWF5YTHNKKMXQH542QEZBO7RGQ: 16
```

AFDHTBSFETGIVMU733ZGGTY63EEA: 16
AEO3CZ4KPIA3I47EADFS43KVFX2A: 16
AGZEDKJEUXQ3KK5D462HQZYTQWXA: 16
AFYONFWGXVIWRVF2SHUIPM6F2C3A: 16
AGBCLRMS7OW3N3T44GO6W6M6RNBA: 16
AFSUK4CHQFQYHUISLHWFDI3T7EGA: 16
AHICFG7RTZ5XSIGBUFK5XVOLZIJQ: 16
AHA4V44CE4KWD2NFAN7RAB4HJJZA: 16
AGJAAVMYOSWYRC3HRWGAWHJFS4WA: 16
AEESX3HNU65OOFXP7VYMTQG5OZLA: 16
AHHTZKWLL3I3MRXLE2RD5T32OA5Q: 16
AE42Q4LRELZEBXTDYI6PYTWZEOZQ: 16
AELNRQWRJQYTAKSHAXR3LBUGIFGA: 16
AG7RRPB7KFXRKFIUEFLCLZ2KU23Q: 16
AF5KJIMQXNEY7M5U5YM43QPCO4PA: 16
AGJX6XVM4R33WWI4R4PNSXVFZ7RA: 16
AGWAUKTSERZ3O4NYZWZS7OK5ABPQ: 16
AFVXOXPDFHFXLCYM6H2HM5PUOCMQ: 16
AGU6C6L32G5WTFF334TTCEWIGGVA: 16
AGTGRAU2XMEYIX4WPQPNY3JGAUNQ: 16
AFWITPOXSUXLPPPZTIHDHV7QZIZQ: 16
AEJVWNXW55ISA3OATOA64QX5WXYQ: 16
AFAZC54MQKYVDQH7NY4GJ7BDBCMQ: 16
AGNZ55VEXGISWO3OH7F4CTP6HNRA: 16
AGIBZ4QTWKA34IT3EVMH52FKJERQ: 16
AEF2RX7ZF5KKRXYURVRY6AKZGLSQ: 16
AEBGYELMA4BKQPPWGIREPQXDLGQA: 16
AEBD7F5ITOHM4VSB4V7L2G35YBSQ: 16
AFB6OB3NPQ3ZCIFC7CQG6JL6J7DQ: 16
AEIGNQD2PURKBD57LI4CYPUF2SNA: 16
AESDNHQRRZEWTZF7A7TFFFTSNY5Q: 17
AFGMXXQWDSRK4H7UWEFZKF6O3JOQ: 17
AH3UBUKNUPJRPZ4MFG4JFE2BD2RA: 17
AHWXEOF3AV66R44YXMV6OL3GTLWQ: 17
AGDR5UXGHUY2HYK276OUE7J4T73Q: 17
AG23AW56WZ7KPYYAS4A3YUDTYWCQ: 17
AFEPLHWMMPUHJCXBCDWHIDLE2A6Q: 17
AHXZMBZM7TCTGR6YQ7MI7LEAZ7QA: 17
AE37RAW77LNOTEDKMDKGXSGQHD5Q: 17
AHZER36Y6IL7USHGBCELVMN56VOQ: 17
AGY53AAWLQK5O77H2YKXCURQHNTQ: 17
AFJAAQX3CJRKW3ETBKBU4XTHMQKA: 17
AGFCZHTATBUI4HMETZZ5DXULTZJQ: 17
AGOLVETLIB3P4APNHIIKQXZB4Y5Q: 17
AGMTCMP27WCMMXZ3R3FJ5F4KDABA: 17
AG43KQOKOO573AON6M3W5TYWWE6Q: 17
AGFQKJKXKIMUYSK462UTSDU2Q7QQ: 17
AHQIY3JCKATNBTZVZAQLHPDE5KTA: 17

AH3XWQAKGWZVH2IZ2TZYUZLGGWZQ: 17
AGTVZ7ZSDMTEDLMJGZRC7RFJNDWQ: 18
AHJPWPHCWZ5MX2UY6CNGJUNBWP3A: 18
AHX2B4DEER2QR3IU3CCNB3CWC6TA: 18
AHVTPXABLBUBOHIQYQ3QQG6WR6LA: 18
AFKBZVPC42DGIECDEAOCL57MRAQA: 18
AE23ZFVUOMPKR57BVSWXV34QLMVA: 18
AGAAUEJ3GP5HNRCP54UMGKPNWVUA: 18
AEHPI7DGK45GM4JO4PP25BE3RZJQ: 18
AF5MQN46S6QJ3R7F5TEXLMDML66Q: 18
AGK24I67HJO562FE5RKCGI24T3HA: 18
AE6GRTPFO4YSJUB42J5ID6NUQ26A: 18
AHGYSJ5C6F7YUTGNX6EMVI5QLYUA: 18
AGUG7ONCKL4JOYS6AILUT3AGOCWQ: 18
AEY67UM6R2R2XFEAXAXMWTZ4Z5UA: 18
AGG2QQ2TPFEBHWNTVYO5ICVDGHCQ: 18
AE3MLVJ2OOZ3WLVKQG42G3AQ2KZQ: 19
AHOBKBVHEAT4KPRVI7TYTF6MEJXA: 19
AETEDS6HQJWX2EQYEQFUIAHTTJXQ: 19
AEQWIQ5K6LLNVFPBJTPKK4J76SWA: 19
AETAVHERYX7O3ND5SFSQ3UZNQ6LQ: 19
AGJ4GTDNTPHMRM26V46EYAF7IGCA: 19
AEPGWR4J3CQI76JJVCCOQQJEH2CA: 19
AGIOGJDBPAJSQTMQRYQMNUVRZ6OA: 19
AHIPKIT25JKKXMUL332SVAWYAKBA: 19
AERE4WE4ZXZUK2TWOR33P6XWHPHQ: 19
AGXCV25HDW4JXI5QQSJTIZUHPRHQ: 20
AHSLFXG4FNPYB567N4G2TZH6SK5A: 20
AH26RPVEVUPMVDCLOOFLVAIOYHFQ: 20
AGXH2GDBRITH4C3BCMUJY2H4LCIA: 20
AGUQUN4TIWP46MPNHP5QOPOPRFPA: 20
AFFMTXZ6ZKG7PV3BO6UKZBSLXYPA: 20
AFEG7SPWPJAFUDLSOGA3QNU6XELA: 20
AEAL4MJGXLIH3VY2GEZJUT54KILA: 20
AGWYFSFSBGELWEVIJSTNQFP5QOZQ: 20
AH2KNTQFU4DWMQIMTDLFSMRQHRYA: 20
AHEJYPLN7DMCYRQ2KCWQ5YIY6JDA: 21
AGX2NNH2M3RNS5SODCZMONPR2PAA: 21
AG3S4FROO422V5KP7DJCBXVUQLJQ: 21
AGUEUIXS3ETYDF3UKLAP42NXEYWQ: 21
AHGDGGMCSMMALHTX6WGLJFFUVJXA: 21
AE7GVDNRY5NABEP253Y3GMDUZOZA: 21
AG6EDKW4LTUP3EKVJUFQWJTF3AYA: 21
AF2TXYUSL64QAUAGQ5H54JHCGSKA: 21
AF7DTXFZKRXELYXQKP54UAHBPDDA: 21
AHLFYDRLX6L6LLJ4L25C5XFNEWKA: 21
AFQGYW6TLQ7IOG475GG6WDXHPY5Q: 21
AFPDU4XGTLCRX5E6OUTZQFDALVTQ: 21

AEB5MUKIPEWYQ6TIWV2WQJK3XVYA: 21
AHQKCRAIW6KOCBXNQQ2HCMDHPPCQ: 21
AGKIBYKFLBFAZ2LELCHNWU3JKPCA: 22
AHCIO4Y7MASS5GHGAZNOPPTXFTVA: 22
AFMOXNCYTYVRTZKJGVX4X7NHOU7Q: 22
AG5BM4MB6I32TDPE2SCSQXSCJCHQ: 22
AGBDLSGHRFYWVTPVFFMGGT3W4NNA: 22
AGH7F3VLUYMLE52O7FHD7X2B5RAA: 22
AFBFEQ63CH3PADWCT52FIYQLG7XA: 23
AETDJT3WINHMRGVYOPS7SF7KLHVQ: 23
AG26EFNGF4Y2N3LC7XZRNLTG4P4A: 23
AEWFRZ4QAEFDQEBPTVMJ6OOSIH2Q: 23
AFFELFYTX2FJCTZLASZLK3K7FQNA: 23
AEVIDOETQTYGFWUVJGTIDMWH4YFQ: 24
AERJUD4NREPMW7VM26CSV32KJZJQ: 24
AEJSQT3KBWNDGDU2NJIEPKNTGLMA: 24
AHMY4433RXWWWIPLL52FFK3KA2BA: 24
AG42JLA6HBMONDJECN3JLAUY3JXA: 25
AECRFIY3HZZG5KMU424HTLAKDGFA: 25
AF4H7MSDHSUVEMA2BPLP6MWGITGQ: 25
AG44UYFDZHI6FRBQSLDWOGIEOY4A: 26
AHDJSYNKXNWW2MQQPABYYMH7RTRQ: 26
AEKHOFU35JS3BOIWZSEM7DKSTEVA: 27
AHMQXXMPUH52LC6PKXH4YAJT2FDQ: 27
AHO5IPKZNXFFG2LYSCTINORZG7EQ: 27
AGBWDIAHWZGV2SIYX2KDCBB6746A: 27
AEF57CDFWDBRC4QJ3RPLBAHB2JGA: 27
AG3PQBC2VIMREI2WXU7IFYRIGTPQ: 28
AFYQUX7UGA5EIGT6SIU6GT5Q67MQ: 29
AF3NTN5FD5C3WWBBL4EYNZPA7GVA: 29
AEHWHU26Z2VEZQY3IEEW5HBIGX5Q: 30
AHLW46IEEWKFIKZQIWJME267TRZQ: 30
AEKUZFUOA5Y7YXZTA3W3L3SHLQCQ: 32
AH4GZFH3BWFUJTLJ7P4QEFSWGZYA: 32
AHEMAURZYRCARVTVF6CQYTYE4I4Q: 32
AFWEOWK32WTIBX4MC7D3WFDLCKJQ: 33
AFDAU5M5NRUN4LYLI7N4YROKYLWA: 34
AHNG3NRGEDYO7E44QMYDHMBADPBA: 35
AHJQIRE3KBRMPLVEGQNUOTYILZDA: 36
AG6ATLCKETNYRKL6GCQV35CSSGPA: 36
AHR6CSQFJVG5AHI25CP4NTJDEZFQ: 38
AG4ZLTRHVAZRU33BPY5Y643IZXPQ: 40
AGVT5E2C6WNOCFYSLOCPAPBJOIKQ: 41
AG5ZVXXHEXDYUUODSEQC4XXV7DPA: 43
AFJQQWO6VAZCTTY3GXNB3SEUU34Q: 43
AF7CC34DK36SQJS7WXI44DREGWJA: 48

Compute Hit rate on the test set for the 2 groups of users and discuss the difference (or similarity)

between those scores.

[124]:
```
# Restrict ground_truth to the two user groups (from cell 20)
gt_top = {user: items for user, items in ground_truth.items() if user in␣
 ↪top_users}
gt_bottom = {user: items for user, items in ground_truth.items() if user in␣
 ↪bottom_users}

# Compute the metrics for each group using the existing evaluate function.
# Here, we extract the hit rate (index 0 of the returned tuple).

svd_top_metrics = evaluate(svd_recs, gt_top, k=10, total_items=total_items)
svd_bottom_metrics = evaluate(svd_recs, gt_bottom, k=10,␣
 ↪total_items=total_items)
print("SVD Hit Rate for Top 20% Users:", svd_top_metrics[0])
print("SVD Hit Rate for Bottom 20% Users:", svd_bottom_metrics[0])

# Compute metrics for KNNWithMeans on top and bottom users.
knn_top_metrics = evaluate(knn_recs, gt_top, k=10, total_items=total_items)
knn_bottom_metrics = evaluate(knn_recs, gt_bottom, k=10,␣
 ↪total_items=total_items)
print("KNN Hit Rate for Top 20% Users:", knn_top_metrics[0])
print("KNN Hit Rate for Bottom 20% Users:", knn_bottom_metrics[0])

# Compute metrics for TopPop on top and bottom users.
top_pop_top_metrics = evaluate(top_pop_recs, gt_top, k=10,␣
 ↪total_items=total_items)
top_pop_bottom_metrics = evaluate(top_pop_recs, gt_bottom, k=10,␣
 ↪total_items=total_items)
print("TopPop Hit Rate for Top 20% Users:", top_pop_top_metrics[0])
print("TopPop Hit Rate for Bottom 20% Users:", top_pop_bottom_metrics[0])
```

```
SVD Hit Rate for Top 20% Users: 0.09836065573770492
SVD Hit Rate for Bottom 20% Users: 0.11949685534591195
KNN Hit Rate for Top 20% Users: 0.18032786885245902
KNN Hit Rate for Bottom 20% Users: 0.14465408805031446
TopPop Hit Rate for Top 20% Users: 0.16393442622950818
TopPop Hit Rate for Bottom 20% Users: 0.3710691823899371
```

Sort the items by number of ratings in the train set and select the top 20% and last 20% of items.

[125]:
```
# Calculate the number of ratings for each item in the trainset
item_rating_counts = {}
for inner_iid, user_ratings in trainset.ir.items():
    raw_iid = trainset.to_raw_iid(inner_iid)
    item_rating_counts[raw_iid] = len(user_ratings)

# Sort items by the count of ratings (ascending order)
```

```python
sorted_items = sorted(item_rating_counts.items(), key=lambda x: x[1])
num_items = len(sorted_items)
num_20_percent = int(num_items * 0.2)

# Select bottom 20% (items with fewest ratings)
bottom_20_items = [item for item, count in sorted_items[:num_20_percent]]

# Select top 20% (items with most ratings)
top_20_items = [item for item, count in sorted_items[-num_20_percent:]]

print("Total number of items:", num_items)
print("Bottom 20% items:", bottom_20_items)
print("Top 20% items:", top_20_items)
```

```
Total number of items: 509
Bottom 20% items: ['B07KRZ1HH8', 'B0BNCDVD4M', 'B0B2LMZ9RT', 'B0C5CTX5LZ',
'B07DVX841V', 'B075MCRFYZ', 'B08Y6PVK2L', 'B073RTT48C', 'B08PBNXZYP',
'B0B63DPVDK', 'B00GGMWA5O', 'B07YMGPV7P', 'B0BGQZNQ53', 'B09JYJYK1N',
'B00ABLI0IC', 'B01LEZVYZO', 'B09Y5NZ8PN', 'B07BFZ2KSJ', 'B0C6HXYY3R',
'B07M6B3RDW', 'B07MDT5WJX', 'B01FM8IN3U', 'B01HUCYOXE', 'B078JQV1DN',
'B07HGRFG5J', 'B07BRK2M2Q', 'B09Y1X5BSC', 'B0BFR5LN7M', 'B0BT2Q355P',
'B086PLHS9L', 'B00X7DDGBW', 'B09B45DLQG', 'B0002MSQVQ', 'B09NLV5LBK',
'B0BJF97MDV', 'B0776RBZF1', 'B00CIHB8FY', 'B07RLVH6H4', 'B08T9CCXZB',
'B07ND5PQTP', 'B00HU25WSQ', 'B0192JRA2A', 'B07VPLFNYM', 'B01M5CXH65',
'B01LYJMH9P', 'B0BJ8WDC13', 'B09QZJ57YW', 'B01LKYTQOK', 'B0777KT21W',
'B0C1Y5KJPX', 'B0035LCFRW', 'B000EEL50Q', 'B09G5KDVWW', 'B07Y94MSG9',
'B000PR3JEM', 'B0002D0MI0', 'B0158GWWJ2', 'B0085545YO', 'B0BJ3BBTXH',
'B09JMYM88H', 'B00724YM7E', 'B092LF6RLS', 'B00LYFFW9O', 'B01LL8PRIY',
'B01LZBHHC2', 'B076ZSHQ47', 'B0B81FBRNV', 'B0756P1DV8', 'B079NS31NK',
'B09PVPJVN8', 'B06XKPHXSV', 'B00BESRB8Q', 'B0B52FBC66', 'B0BQ4HSKC9',
'B00YJ5C46K', 'B00W7ZL4TM', 'B00GK75WXC', 'B07B16JL73', 'B006MVECVE',
'B00IPHISW2', 'B076VWK8JV', 'B07RYQ1X42', 'B0B8P1YW7L', 'B0B8Z2T4PS',
'B01JIQ36DE', 'B0C3QWRH66', 'B0BXT6TM6N', 'B0BT3B7GTY', 'B07KDYH2DF',
'B074QKLW4N', 'B09PJ3N73Y', 'B000TGSM6E', 'B0002D0Q2W', 'B0009DXEEM',
'B09M7DPHCS', 'B0BLLKKFR8', 'B07B4S63NN', 'B0002KZEAE', 'B0B8Z6YPWN',
'B08VJC3GCM', 'B079Y9L1G1']
Top 20% items: ['B078KTV8L4', 'B0BR2G8WYG', 'B015WUVA5G', 'B07SZLTL1W',
'B0BT2ZRCY2', 'B075MPFFRV', 'B09PBL98Q4', 'B09YDBKT7M', 'B0BPKTYTPQ',
'B08DM9NFLH', 'B096K2LZKH', 'B09KWWS7DH', 'B015QK3GUO', 'B01DXVB4RU',
'B06X9KG3HW', 'B0BXT384GR', 'B00JL5I61A', 'B001L8NGJ2', 'B07V46KRD8',
'B00HZH4D04', 'B0C61YVW2S', 'B07N2HQ1T7', 'B07226MCY6', 'B07JGCQYN8',
'B01ANNNKMI', 'B01MRJUB7Z', 'B09396NY1C', 'B0BH599FJY', 'B0BFKQ9QXD',
'B000RNB720', 'B0010SHU18', 'B06W2NS7RW', 'B07D5W5X3Z', 'B07MHL48V6',
'B0BPKH4HB2', 'B07N5J3GTB', 'B00CPLODUU', 'B015X3CXXA', 'B07TDNWYSB',
'B0B8F6LD9F', 'B06XB3FQKB', 'B07GFH8NTB', 'B08DBDD6CM', 'B09KR857F4',
'B0BFKKVQZT', 'B0B8Z76J32', 'B0B8M5FJ9W', 'B00NARHNCS', 'B091MZPYBQ',
'B005M0CK9M', 'B008L3SXE8', 'B09S2VB3KW', 'B00H4PEMM6', 'B097FMTZCP',
'B0BK5CXZBT', 'B01DK40R3M', 'B0B92LPGD2', 'B098KXQJVY', 'B095XZJ99J',
```

```
'B00CGFRJ2Y', 'B0BG95DG2H', 'B097Q9W8MW', 'B015IJIO5U', 'B07CBT4SYD',
'B08BW2TBD7', 'B09PWCF6T9', 'B09M7P8SBH', 'B008BPI2OW', 'B012VQ5A7S',
'B09BF8XDF4', 'B01DE4DTAG', 'B079TLFL33', 'B0C5PRQ9K6', 'B01DE4DPSC',
'B00RVE9X06', 'B074WXM13T', 'B00WHLLDWO', 'B0BKZ3GCGR', 'B00H35YIJE',
'B09RJZSQML', 'B00646MZHK', 'B0928HW2P4', 'B01DPCONFM', 'B00IFOTSJW',
'B0B8M2Q1DS', 'B0BHSG5C3G', 'B0BKR2ZM9X', 'B0BPLFP5P6', 'B0179A4ILK',
'B01K1EJQ9U', 'B08JMQR2JK', 'B09V91H5XM', 'B004XNK7AI', 'B0B95V41NR',
'B08SJY4T7K', 'B0BTC9YJ2W', 'B08R5GM6YB', 'B0BCK6L7S5', 'B09857JRP2',
'B0BPJ4Q6FJ', 'B0BSGM6CQ9']
```

Compute Coverage on the test set for the 2 groups of items and discuss the difference (or similarity) between those scores.

[126]:
```python
# Function to compute coverage for a given recommender's recommendations.
def compute_coverage(recs, group_top, group_bottom, model_name):
    rec_top = set()
    rec_bottom = set()
    for user in test_users:
        rec_list = recs.get(user, [])
        rec_top.update(set(rec_list).intersection(group_top))
        rec_bottom.update(set(rec_list).intersection(group_bottom))
    coverage_top = len(rec_top) / len(group_top) if group_top else 0
    coverage_bottom = len(rec_bottom) / len(group_bottom) if group_bottom else 0
    print(f"{model_name} Coverage for Top 20% Items: {coverage_top:.4f}")
    print(f"{model_name} Coverage for Bottom 20% Items: {coverage_bottom:.
 ↪4f}\n")

# Compute and print coverage for each model.
compute_coverage(svd_recs, top_20_items, bottom_20_items, "SVD")
compute_coverage(knn_recs, top_20_items, bottom_20_items, "KNNWithMeans")
compute_coverage(top_pop_recs, top_20_items, bottom_20_items, "TopPop")
```

```
SVD Coverage for Top 20% Items: 0.1881
SVD Coverage for Bottom 20% Items: 0.0198

KNNWithMeans Coverage for Top 20% Items: 0.4752
KNNWithMeans Coverage for Bottom 20% Items: 0.6238

TopPop Coverage for Top 20% Items: 0.0990
TopPop Coverage for Bottom 20% Items: 0.0000
```

Error analysis for the neighborhood-based CF Select two users from the training set, one with high RR and one with low RR. These are your reference users. Retrieve the 10 nearest neighbours of each reference user. Print their rating history and analyse their predictions. How much overlap is there between the rating history of the reference users and their neighbours?

For those users or items that your model performs poorly on (RR 0.05), discuss the potential reasons behind.

[127]:
```
# Compute per-user Reciprocal Rank (RR) for the KNN model using the␣
 ↪recommendations in knn_recs
user_rr = {}
for user, true_items in ground_truth.items():
    rec = knn_recs.get(user, [])
    rr = 0
    for i, item in enumerate(rec):
        if item in true_items:
            rr = 1.0 / (i + 1)
            break
    user_rr[user] = rr

# Sort users by RR (highest first)
sorted_rr = sorted(user_rr.items(), key=lambda x: x[1], reverse=True)
print("Top 5 users with highest RR:", sorted_rr[:5])
print("Top 5 users with lowest RR:", sorted_rr[-5:])

# Select one reference user with highest RR and one with lowest RR
high_rr_user = sorted_rr[0][0]
low_rr_user = sorted_rr[-1][0]
print(f"\nSelected high RR user: {high_rr_user} (RR = {user_rr[high_rr_user]})")
print(f"Selected low RR user: {low_rr_user} (RR = {user_rr[low_rr_user]})")

# Function to get the k nearest neighbors (using knn_best of the␣
 ↪neighborhood-based model)
def get_neighbors(user_id, k=10):
    inner_uid = trainset.to_inner_uid(user_id)
    neighbor_inner = knn_best.get_neighbors(inner_uid, k=k)
    return [trainset.to_raw_uid(nid) for nid in neighbor_inner]

high_rr_neighbors = get_neighbors(high_rr_user, k=10)
low_rr_neighbors = get_neighbors(low_rr_user, k=10)
print(f"\nHigh RR user neighbors: {high_rr_neighbors}")
print(f"Low RR user neighbors: {low_rr_neighbors}")

# Function to obtain the rating history of a user from the training set
def get_rating_history(user_id):
    inner_uid = trainset.to_inner_uid(user_id)
    # trainset.ur is a dict: key is inner_uid, value is list of (inner_iid,␣
 ↪rating)
    history = trainset.ur[inner_uid]
    return [(trainset.to_raw_iid(iid), rating) for iid, rating in history]

# Print rating history for each reference user and their neighbors
print("\nHigh RR user rating history:")
high_rr_history = get_rating_history(high_rr_user)
print(high_rr_history)
```

```
print("\nHigh RR user neighbors' rating histories:")
for neighbor in high_rr_neighbors:
    print(f"Neighbor {neighbor}: {get_rating_history(neighbor)}")

print("\nLow RR user rating history:")
low_rr_history = get_rating_history(low_rr_user)
print(low_rr_history)

print("\nLow RR user neighbors' rating histories:")
for neighbor in low_rr_neighbors:
    print(f"Neighbor {neighbor}: {get_rating_history(neighbor)}")

# Overlap analysis: for each neighbor compare the set of rated items with the␣
 ↪reference user.
def compute_overlap(ref_history, neighbor_history):
    ref_items = set(item for item, _ in ref_history)
    neighbor_items = set(item for item, _ in neighbor_history)
    common = ref_items.intersection(neighbor_items)
    return len(common), len(ref_items), len(neighbor_items), common

print("\nOverlap analysis for high RR user:")
high_total_overlap = 0
for neighbor in high_rr_neighbors:
    overlap, ref_count, neigh_count, common_items =␣
 ↪compute_overlap(high_rr_history, get_rating_history(neighbor))
    print(f"Neighbor {neighbor}: {overlap} common items (Reference rated␣
 ↪{ref_count}, Neighbor rated {neigh_count})")
    high_total_overlap += overlap
high_avg_overlap = high_total_overlap / len(high_rr_neighbors)
print(f"\nAverage overlap for high RR user with neighbors: {high_avg_overlap:.
 ↪2f}")

print("\nOverlap analysis for low RR user:")
low_total_overlap = 0
for neighbor in low_rr_neighbors:
    overlap, ref_count, neigh_count, common_items =␣
 ↪compute_overlap(low_rr_history, get_rating_history(neighbor))
    print(f"Neighbor {neighbor}: {overlap} common items (Reference rated␣
 ↪{ref_count}, Neighbor rated {neigh_count})")
    low_total_overlap += overlap
low_avg_overlap = low_total_overlap / len(low_rr_neighbors)
print(f"\nAverage overlap for low RR user with neighbors: {low_avg_overlap:.
 ↪2f}")
```

```
Top 5 users with highest RR: [('AHGDGGMCSMMALHTX6WGLJFFUVJXA', 1.0),
('AHN535MBJTI4BGLTUORFR3TLABLA', 1.0), ('AFJ52QWR6Y5WPXK3MEZJJBHMUY7A', 1.0),
```

```
('AFMOXNCYTYVRTZKJGVX4X7NHOU7Q', 1.0), ('AH2KNTQFU4DWMQIMTDLFSMRQHRYA', 0.5)]
Top 5 users with lowest RR: [('AG5ZVXXHEXDYUUODSEQC4XXV7DPA', 0),
('AFKZTCN4BNEKHAJQARBTQ4RZVETQ', 0), ('AH5MZVAOBU2PWPJ2G663BY5S2R5A', 0),
('AEIRBLA7WN4R3DFFWOUBDY6UYWEA', 0), ('AENTQR2OEQO7IPIVUMHMV545FPJA', 0)]

Selected high RR user: AHGDGGMCSMMALHTX6WGLJFFUVJXA (RR = 1.0)
Selected low RR user: AENTQR2OEQO7IPIVUMHMV545FPJA (RR = 0)

High RR user neighbors: ['AE5M7M2VYMM3WHJIBLCHHEU7UOXQ',
'AESDNHQRRZEWTZF7A7TFFFTSNY5Q', 'AHEJYPLN7DMCYRQ2KCWQ5YIY6JDA',
'AG4VWLJ2X7PIBYDXYYUNFY5ONDRQ', 'AEBXDQP3DYUQMQUTAUIMYMOMSRZQ',
'AEDKV2KYMJZD43MAADUEKE7Z3PPA', 'AHJPWPHCWZ5MX2UY6CNGJUNBWP3A',
'AEOOBBNDQVLYBDVJLE4GJUCWX42A', 'AGXCV25HDW4JXI5QQSJTIZUHPRHQ',
'AENYK6CKXCHJDY2YV3WIQZMJ3HNQ']
Low RR user neighbors: ['AE7P3HIBI3UDLCJLUPUZQWYVPEWA',
'AESDNHQRRZEWTZF7A7TFFFTSNY5Q', 'AGX2NNH2M3RNS5SODCZMONPR2PAA',
'AEKUZFUOA5Y7YXZTA3W3L3SHLQCQ', 'AGXCV25HDW4JXI5QQSJTIZUHPRHQ',
'AEYJZUNG4OSYKDPN7WH3LKTBKH3Q', 'AFVLYQWBCIHXVLH6X2FSXRMAHHXA',
'AH5MZVAOBU2PWPJ2G663BY5S2R5A', 'AER6WLSJPBPNOZIQWVHQBVOKTNCA',
'AFBIB5XMB27ZIUM3YDTESOKSLLQA']

High RR user rating history:
[('B00646MZHK', 5.0), ('B0085545YO', 5.0), ('B08R5GM6YB', 5.0), ('B000EEL50Q',
4.0), ('B01DECWM0G', 1.0), ('B008BPI2OW', 5.0), ('B01DE4DEGK', 5.0),
('B0BFKKVQZT', 5.0), ('B0BNVFLP64', 5.0), ('B095XCZXNQ', 3.0), ('B09857JRP2',
5.0), ('B07FZ5XBCJ', 5.0), ('B00GGMWA5O', 5.0), ('B00YJ5C46K', 5.0),
('B0027V760M', 5.0), ('B0150YC54E', 4.0), ('B00X7DDGBW', 5.0), ('B01J5A06OU',
5.0), ('B09JYHK3KF', 5.0), ('B007MY5BDI', 5.0), ('B07KDYH2DF', 5.0)]

High RR user neighbors' rating histories:
Neighbor AE5M7M2VYMM3WHJIBLCHHEU7UOXQ: [('B000KIPTE4', 5.0), ('B005M0CK9M',
5.0), ('B005M0MUQK', 5.0), ('B007T8OGLK', 5.0), ('B097Q9W8MW', 4.0),
('B09JYHK3KF', 5.0)]
Neighbor AESDNHQRRZEWTZF7A7TFFFTSNY5Q: [('B0C67HCGBR', 5.0), ('B00HZH4D04',
4.0), ('B0C61YVW2S', 5.0), ('B0BNVFLP64', 4.0), ('B0B95V41NR', 4.0),
('B00HU260SW', 4.0), ('B08SJY4T7K', 4.0), ('B000T9L7W2', 5.0), ('B06W2NS7RW',
4.0), ('B0BPJ4Q6FJ', 5.0), ('B07DK59QNR', 5.0), ('B07N2HQ1T7', 5.0),
('B07D5W5X3Z', 4.0), ('B07NP7HF7X', 4.0), ('B00JL5I61A', 4.0), ('B016QUXTZU',
4.0), ('B0BZ1XQX97', 4.0)]
Neighbor AHEJYPLN7DMCYRQ2KCWQ5YIY6JDA: [('1423414357', 5.0), ('B07JCW1KGG',
5.0), ('B000T9L7W2', 5.0), ('B000RNB72O', 5.0), ('B0BR2Q367S', 5.0),
('B0BPJ4Q6FJ', 5.0), ('B0BPKTYTPQ', 5.0), ('B005T800V2', 5.0), ('B00646MZHK',
5.0), ('B074KRRZCH', 3.0), ('B01MRJUB7Z', 5.0), ('B0BFKKVQZT', 5.0),
('B09PWCF6T9', 5.0), ('B07B16JL73', 5.0), ('B081592T7F', 5.0), ('B00DKAQGTG',
5.0), ('B095QBH5ZB', 5.0), ('B07DWM2XSS', 5.0), ('B07PM289SV', 5.0),
('B00D35CNL8', 5.0), ('B0817CVBLV', 5.0)]
Neighbor AG4VWLJ2X7PIBYDXYYUNFY5ONDRQ: [('B0C61YVW2S', 4.0), ('B0BKZ5F8BS',
5.0), ('B09BR11LYJ', 5.0), ('B0BCK6L7S5', 5.0), ('B019NY2PKG', 5.0),
```

('B07FZ5XBCJ', 5.0), ('B07N5J3GTB', 5.0)]
Neighbor AEBXDQP3DYUQMQUTAUIMYMOMSRZQ: [('B000TGSM6E', 2.0), ('B07DWLYGKH',
5.0), ('B003BGXW50', 4.0), ('B01CSAATY6', 4.0), ('B078L11275', 5.0),
('B09857JRP2', 3.0), ('B097FMTZCP', 4.0), ('B0002D0Q2W', 2.0), ('B0B8F6LD9F',
5.0), ('B076ZSHQ47', 4.0), ('B095QBH5ZB', 5.0)]
Neighbor AEDKV2KYMJZD43MAADUEKE7Z3PPA: [('B000T9L7W2', 3.0), ('B0C61YVW2S',
5.0), ('B0B8M2Q1DS', 5.0), ('B0BTC9YJ2W', 5.0), ('B0002E3FBA', 5.0),
('B01DE4DEGK', 5.0), ('B0BSGM6CQ9', 4.0), ('B091MZPYBQ', 5.0), ('B07N5J3GTB',
5.0), ('B08R5GM6YB', 5.0), ('B06XB3FQKB', 5.0), ('B00OYUSHJS', 4.0)]
Neighbor AHJPWPHCWZ5MX2UY6CNGJUNBWP3A: [('B0BPJ4Q6FJ', 4.0), ('B004U1QDL0',
4.0), ('B06XB3FQKB', 2.0), ('B00G964BUY', 5.0), ('B0BXT41XC6', 5.0),
('B07M69RBH7', 5.0), ('B0150YC54E', 5.0), ('B004XNK7AI', 5.0), ('B0B8YX1H2H',
5.0), ('B00CIO6BEA', 2.0), ('B0BT2W3TTM', 5.0), ('B00WT7KPCK', 5.0),
('B0C5PRQ9K6', 5.0), ('B07NKMZJ5M', 5.0), ('B0BSR996X8', 4.0), ('B005M0KLGQ',
3.0), ('B09PBL98Q4', 3.0), ('B0BCK6L7S5', 5.0)]
Neighbor AEOOBBNDQVLYBDVJLE4GJUCWX42A: [('B0BFKKVQZT', 4.0), ('B08R5GM6YB',
4.0), ('B08SJY4T7K', 4.0), ('B07XP6MHQT', 5.0), ('B0010SHU18', 3.0),
('B07V46KRD8', 5.0), ('B01CS87RGQ', 5.0), ('B0BXT384GR', 2.0), ('B00IPH8MD2',
5.0), ('B0B95V41NR', 3.0), ('B01NAZMWZK', 1.0), ('B07N5J3GTB', 5.0),
('B00IZA1GI2', 5.0)]
Neighbor AGXCV25HDW4JXI5QQSJTIZUHPRHQ: [('B0002D0Q2W', 5.0), ('B079P9LDHN',
5.0), ('B00IFOTSJW', 5.0), ('B00JL5I61A', 3.0), ('B01DZ6FBBI', 4.0),
('B09RJZSQML', 5.0), ('B00P6ZOPP0', 3.0), ('B0BPLFP5P6', 4.0), ('B0BCK6L7S5',
4.0), ('B007T8CUNG', 4.0), ('B012VQ5A7S', 5.0), ('B01DK40R3M', 4.0),
('B004U1QDL0', 4.0), ('B004OK1G64', 5.0), ('B01MRJUB7Z', 4.0), ('B09TRX519R',
3.0), ('B081592T7F', 5.0), ('B01DECWM0G', 4.0), ('B0BSGM6CQ9', 5.0),
('B073T2YK2J', 5.0)]
Neighbor AENYK6CKXCHJDY2YV3WIQZMJ3HNQ: [('B00IFOTSJW', 5.0), ('B00646MZHK',
5.0), ('B004XNK7AI', 5.0), ('B07CBT4SYD', 5.0), ('B0BYXVCSGG', 5.0),
('B06Y6BCX7B', 5.0), ('B0B3VSZQHL', 5.0), ('B079P9LDHN', 5.0), ('B000CZ0RKG',
5.0), ('B00UTD0YG8', 5.0)]

Low RR user rating history:
[('B09TRX519R', 5.0), ('B00BESRB8Q', 4.0), ('B0BHSG5C3G', 4.0), ('B00UTELPGK',
4.0), ('B06W2NS7RW', 4.0), ('B0BT2Z1DF1', 5.0), ('B00HY657E2', 4.0),
('B0B29LK8QD', 5.0), ('B0BLZD2X18', 5.0), ('B014ZYPT34', 5.0), ('B00H35YIJE',
5.0), ('B07KYX3S8X', 5.0)]

Low RR user neighbors' rating histories:
Neighbor AE7P3HIBI3UDLCJLUPUZQWYVPEWA: [('B09BF8XDF4', 4.0), ('B074WXM13T',
5.0), ('B07GFH8NTB', 5.0), ('B00H35YIJE', 4.0), ('B07B4S63NN', 4.0),
('B003AYNHGW', 5.0), ('B000NGVQKO', 4.0), ('B095QBH5ZB', 4.0), ('B07DWM2XSS',
5.0)]
Neighbor AESDNHQRRZEWTZF7A7TFFFTSNY5Q: [('B0C67HCGBR', 5.0), ('B00HZH4D04',
4.0), ('B0C61YVW2S', 5.0), ('B0BNVFLP64', 4.0), ('B0B95V41NR', 4.0),
('B00HU260SW', 4.0), ('B08SJY4T7K', 4.0), ('B000T9L7W2', 5.0), ('B06W2NS7RW',
4.0), ('B0BPJ4Q6FJ', 5.0), ('B07DK59QNR', 5.0), ('B07N2HQ1T7', 5.0),
('B07D5W5X3Z', 4.0), ('B07NP7HF7X', 4.0), ('B00JL5I61A', 4.0), ('B016QUXTZU',

4.0), ('B0BZ1XQX97', 4.0)]
Neighbor AGX2NNH2M3RNS5SODCZMONPR2PAA: [('B079TLFL33', 5.0), ('B00H35YIJE',
4.0), ('B0B95V41NR', 5.0), ('B000RNB720', 3.0), ('B08R5GM6YB', 4.0),
('B07CBT4SYD', 3.0), ('B0B2LSX437', 5.0), ('B0BXT384GR', 3.0), ('B07WW5PKCP',
5.0), ('B0B8Z6GZZ5', 5.0), ('B01DE4DEGK', 5.0), ('B06XB3FQKB', 5.0),
('B005T800V2', 5.0), ('B0BG95DG2H', 5.0), ('B0BKZ3GCGR', 5.0), ('B01DE4DPSC',
5.0), ('B0064RTS0G', 5.0), ('B0C5PRQ9K6', 5.0), ('B00CIO6BEA', 5.0),
('B09V91H5XM', 4.0), ('B00646MZHK', 3.0)]
Neighbor AEKUZFUOA5Y7YXZTA3W3L3SHLQCQ: [('B00H4PEMM6', 5.0), ('B0BR2Q367S',
4.0), ('B0BXT384GR', 4.0), ('B091MZPYBQ', 5.0), ('B001W99HE8', 5.0),
('B097FMTZCP', 5.0), ('B077GH8LF6', 5.0), ('B015X3CXXA', 5.0), ('B00H35YIJE',
5.0), ('B0B95V41NR', 5.0), ('B0BRSM5XBB', 3.0), ('B0756P1DV8', 5.0),
('B09SNBQCM8', 5.0), ('B01DZ6FBBI', 5.0), ('B07GFH8NTB', 3.0), ('B007MY5BDI',
5.0), ('B08R5GM6YB', 4.0), ('B07CBT4SYD', 4.0), ('B0010SHU18', 5.0),
('B01DK43HMK', 5.0), ('B0002D0CEO', 5.0), ('B09S2VB3KW', 5.0), ('B01DE4DTAG',
5.0), ('B0B81FBRNV', 5.0), ('B0BQ4HSKC9', 5.0), ('B00HU260SW', 5.0),
('B09198262S', 5.0), ('B0002D0CNA', 5.0), ('B0BKR2ZM9X', 5.0), ('B09NWCD36N',
5.0), ('B00IPH8MD2', 5.0), ('B08BJGBHN7', 5.0)]
Neighbor AGXCV25HDW4JXI5QQSJTIZUHPRHQ: [('B0002D0Q2W', 5.0), ('B079P9LDHN',
5.0), ('B00IFOTSJW', 5.0), ('B00JL5I61A', 3.0), ('B01DZ6FBBI', 4.0),
('B09RJZSQML', 5.0), ('B00P6ZOPP0', 3.0), ('B0BPLFP5P6', 4.0), ('B0BCK6L7S5',
4.0), ('B007T8CUNG', 4.0), ('B012VQ5A7S', 5.0), ('B01DK40R3M', 4.0),
('B004U1QDL0', 4.0), ('B004OK1G64', 5.0), ('B01MRJUB7Z', 4.0), ('B09TRX519R',
3.0), ('B081592T7F', 5.0), ('B01DECWM0G', 4.0), ('B0BSGM6CQ9', 5.0),
('B073T2YK2J', 5.0)]
Neighbor AEYJZUNG4OSYKDPN7WH3LKTBKH3Q: [('B0B8M2Q1DS', 5.0), ('B0BPJ4Q6FJ',
5.0), ('B07CRK35NG', 3.0), ('B015HG7F3Q', 1.0), ('B06W2NS7RW', 4.0),
('B09V91H5XM', 4.0), ('B06XB3FQKB', 4.0), ('B08R5GM6YB', 5.0), ('B08JMQR2JK',
4.0), ('B07D5W5X3Z', 4.0), ('B0BSGM6CQ9', 4.0), ('B07BHJH2Y8', 5.0),
('B095XCZXNQ', 5.0)]
Neighbor AFVLYQWBCIHXVLH6X2FSXRMAHHXA: [('B07ND5PQTP', 5.0), ('B015XE0B7O',
5.0), ('B00IFOTSJW', 5.0), ('B07CBT4SYD', 2.0), ('B09BF8XDF4', 5.0),
('B07G37S6S9', 5.0), ('B0BSCFTV2G', 5.0), ('B0BK5CXZBT', 4.0), ('B09TRX519R',
5.0)]
Neighbor AH5MZVAOBU2PWPJ2G663BY5S2R5A: [('B00H35YIJE', 2.0), ('B09M7DPHCS',
5.0), ('B0C5PRQ9K6', 5.0), ('B0B81FBRNV', 4.0), ('B0B8Z6192W', 3.0),
('B0002KZEAE', 5.0), ('B07226MCY6', 5.0), ('B08NVVLB16', 5.0), ('B07DWM2XSS',
1.0), ('B09G5KLKX2', 5.0)]
Neighbor AER6WLSJPBPNOZIQWVHQBVOKTNCA: [('B0BK5CXZBT', 5.0), ('B0BPJ4Q6FJ',
5.0), ('B0BKZ3GCGR', 4.0), ('B08R5GM6YB', 5.0), ('B00IFOTSJW', 5.0),
('B0BCXRXTNY', 5.0), ('B0BSGM6CQ9', 5.0), ('B06XB3FQKB', 5.0), ('B07L6RCDP7',
5.0), ('B06ZZCMP8L', 5.0), ('B0BHSG5C3G', 2.0), ('B09PWCF6T9', 5.0)]
Neighbor AFBIB5XMB27ZIUM3YDTESOKSLLQA: [('B00H35YIJE', 2.0), ('B07CBT4SYD',
5.0), ('B07CRK35NG', 5.0), ('B07NKMZJ5M', 5.0), ('B096K2LZKH', 5.0),
('1423414357', 4.0), ('B00646MZHK', 5.0), ('B0002D0CEO', 5.0), ('B093YKLWVN',
5.0), ('B07V46KRD8', 5.0), ('B0BKR2ZM9X', 5.0), ('B09R6KV6QX', 5.0),
('B00HU260SW', 5.0), ('B0C6J1X7TD', 5.0)]

```
Overlap analysis for high RR user:
Neighbor AE5M7M2VYMM3WHJIBLCHHEU7UOXQ: 1 common items (Reference rated 21,
Neighbor rated 6)
Neighbor AESDNHQRRZEWTZF7A7TFFFTSNY5Q: 1 common items (Reference rated 21,
Neighbor rated 17)
Neighbor AHEJYPLN7DMCYRQ2KCWQ5YIY6JDA: 2 common items (Reference rated 21,
Neighbor rated 21)
Neighbor AG4VWLJ2X7PIBYDXYYUNFY5ONDRQ: 1 common items (Reference rated 21,
Neighbor rated 7)
Neighbor AEBXDQP3DYUQMQUTAUIMYMOMSRZQ: 1 common items (Reference rated 21,
Neighbor rated 11)
Neighbor AEDKV2KYMJZD43MAADUEKE7Z3PPA: 2 common items (Reference rated 21,
Neighbor rated 12)
Neighbor AHJPWPHCWZ5MX2UY6CNGJUNBWP3A: 1 common items (Reference rated 21,
Neighbor rated 18)
Neighbor AEOOBBNDQVLYBDVJLE4GJUCWX42A: 2 common items (Reference rated 21,
Neighbor rated 13)
Neighbor AGXCV25HDW4JXI5QQSJTIZUHPRHQ: 1 common items (Reference rated 21,
Neighbor rated 20)
Neighbor AENYK6CKXCHJDY2YV3WIQZMJ3HNQ: 1 common items (Reference rated 21,
Neighbor rated 10)

Average overlap for high RR user with neighbors: 1.30

Overlap analysis for low RR user:
Neighbor AE7P3HIBI3UDLCJLUPUZQWYVPEWA: 1 common items (Reference rated 12,
Neighbor rated 9)
Neighbor AESDNHQRRZEWTZF7A7TFFFTSNY5Q: 1 common items (Reference rated 12,
Neighbor rated 17)
Neighbor AGX2NNH2M3RNS5SODCZMONPR2PAA: 1 common items (Reference rated 12,
Neighbor rated 21)
Neighbor AEKUZFUOA5Y7YXZTA3W3L3SHLQCQ: 1 common items (Reference rated 12,
Neighbor rated 32)
Neighbor AGXCV25HDW4JXI5QQSJTIZUHPRHQ: 1 common items (Reference rated 12,
Neighbor rated 20)
Neighbor AEYJZUNG4OSYKDPN7WH3LKTBKH3Q: 1 common items (Reference rated 12,
Neighbor rated 13)
Neighbor AFVLYQWBCIHXVLH6X2FSXRMAHHXA: 1 common items (Reference rated 12,
Neighbor rated 9)
Neighbor AH5MZVAOBU2PWPJ2G663BY5S2R5A: 1 common items (Reference rated 12,
Neighbor rated 10)
Neighbor AER6WLSJPBPNOZIQWVHQBVOKTNCA: 1 common items (Reference rated 12,
Neighbor rated 12)
Neighbor AFBIB5XMB27ZIUM3YDTESOKSLLQA: 1 common items (Reference rated 12,
Neighbor rated 14)

Average overlap for low RR user with neighbors: 1.00
```

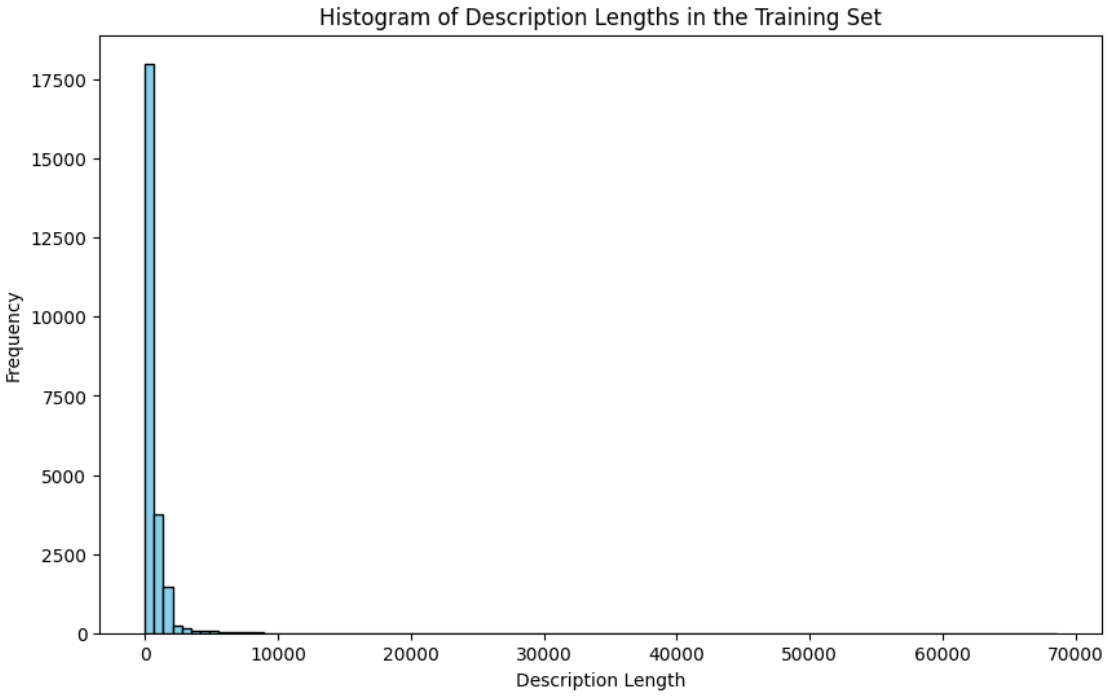
0.0.3 Week 9 [Feb 24 - Mar 2]: Text Representation

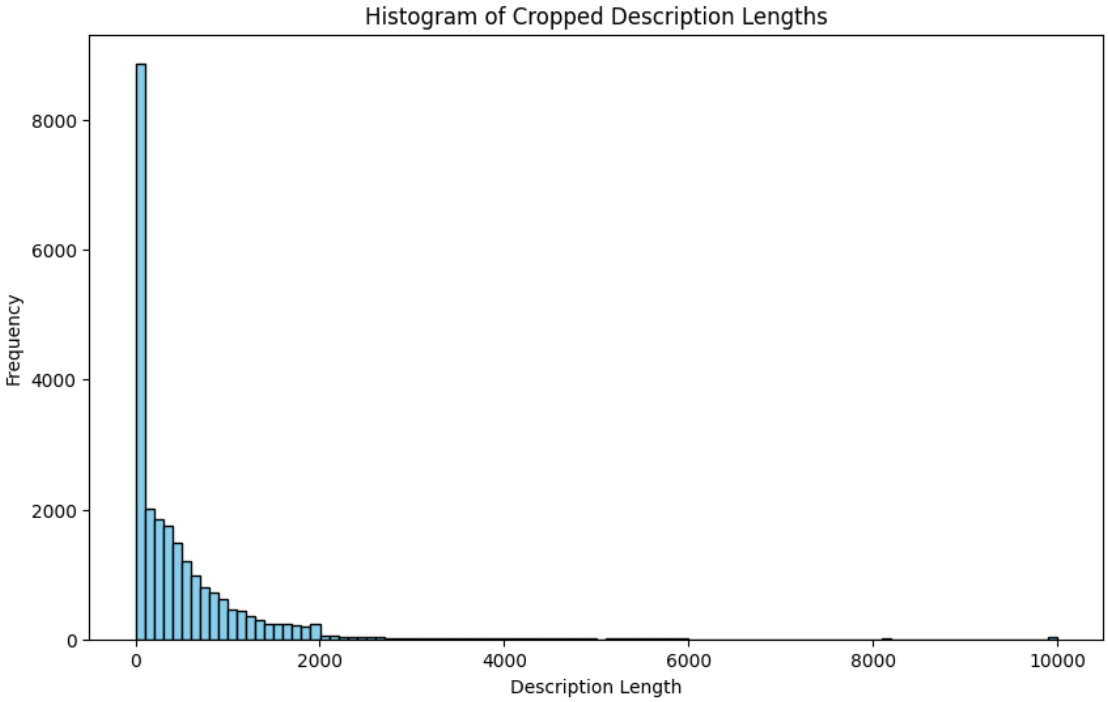
In this part, we will work with the text content from the dataset and will apply NLP techniques to represent items and users in a vector space.

1. Note: for this part, you are free to use all or a subset of the metadata file, e.g., only using items that are in the train/test splits or using the first n characters of the text content. Please justify your choice.

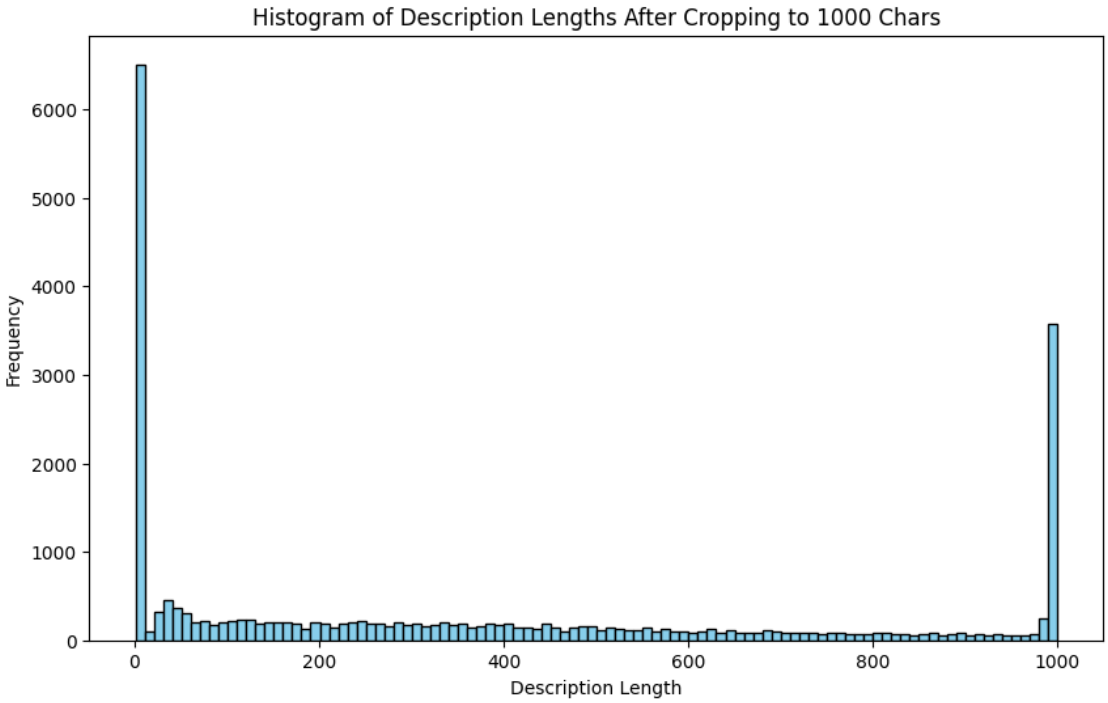
[128]:
```
import matplotlib.pyplot as plt

# Load the metadata containing item descriptions (if not already loaded)
df_meta = pd.read_csv('Data/metadata.tsv', sep='\t')

# Compute the length of each description (convert to string to avoid NaN issues)
df_meta['desc_len'] = df_meta['description'].astype(str).apply(len)

# Plot a histogram of the description lengths
plt.figure(figsize=(10, 6))
plt.hist(df_meta['desc_len'], bins=100, color='skyblue', edgecolor='black')
plt.xlabel('Description Length')
plt.ylabel('Frequency')
plt.title('Histogram of Description Lengths in the Training Set')
plt.show()

# Crop descriptions longer than 10000 characters
df_meta['description'] = df_meta['description'].astype(str).apply(lambda x: x[:
 ↪10000] if len(x) > 10000 else x)
# Recompute description lengths after cropping
df_meta['desc_len'] = df_meta['description'].apply(len)

# Plot histogram of the new description lengths
plt.figure(figsize=(10, 6))
plt.hist(df_meta['desc_len'], bins=100, color='skyblue', edgecolor='black')
plt.xlabel('Description Length')
plt.ylabel('Frequency')
plt.title('Histogram of Cropped Description Lengths')
plt.show()

total_reviews = len(df_meta)
reviews_lt_2000 = (df_meta['desc_len'] < 2000).sum()
reviews_lt_1000 = (df_meta['desc_len'] < 1000).sum()
reviews_lt_500 = (df_meta['desc_len'] < 500).sum()

perc_lt_2000 = reviews_lt_2000 / total_reviews * 100
perc_lt_1000 = reviews_lt_1000 / total_reviews * 100
perc_lt_500 = reviews_lt_500 / total_reviews * 100
```

```
print(f"Percentage of reviews shorter than 2000 chars: {perc_lt_2000:.2f}%")
print(f"Percentage of reviews shorter than 1000 chars: {perc_lt_1000:.2f}%")
print(f"Percentage of reviews shorter than 500 chars: {perc_lt_500:.2f}%")

# Crop descriptions to keep only the first 1000 characters
def crop_text(text):
    # If the string is less than or equal to 1000 characters, return it as is.
    if len(text) <= 1000:
        return text
    # Find the last space before the 1000th character.
    cut_index = text.rfind(' ', 0, 1000)
    # If no space is found, break at 1000 characters.
    if cut_index == -1:
        cut_index = 1000
    return text[:cut_index]

df_meta['description'] = df_meta['description'].astype(str).apply(crop_text)
# Recompute description lengths after cropping
df_meta['desc_len'] = df_meta['description'].apply(len)

# Plot histogram of the new description lengths after cropping
plt.figure(figsize=(10, 6))
plt.hist(df_meta['desc_len'], bins=100, color='skyblue', edgecolor='black')
plt.xlabel('Description Length')
plt.ylabel('Frequency')
plt.title('Histogram of Description Lengths After Cropping to 1000 Chars')
plt.show()
```






```
Percentage of reviews shorter than 2000 chars: 96.50%
Percentage of reviews shorter than 1000 chars: 84.48%
Percentage of reviews shorter than 500 chars: 66.51%
```



2. Select the column description from the metadata file and apply appropriate preprocessing to clean up the data, for example: tokenization, transformation to lowercase, stopword removal6, or stemming. Motivate your preprocessing choices and report the vocabulary size before and after preprocessing. There are many libraries you can use, including but not limited to, NLTK, spaCy or CoreNLP (requires Java).

[129]:
```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import SnowballStemmer
import string

nltk.download('punkt')
nltk.download('stopwords')

# Get the raw descriptions from the metadata DataFrame
descriptions = df_meta['description'].dropna().astype(str).tolist()

# Define stopwords and stemmer
stop_words = set(stopwords.words('english'))
stemmer = SnowballStemmer(language='english')

# Preprocess function: tokenize, lower, remove punctuation, numbers and␣
 ↪stopwords, and stem
```

```
def preprocess_text(text):
    # Tokenize and lowercase
    tokens = [word.lower() for word in word_tokenize(text)]
    # Remove punctuation tokens, stopwords, and tokens containing any digits
    tokens = [token for token in tokens if token not in string.punctuation and␣
 ↪token not in stop_words and not any(ch.isdigit() for ch in token)]
    # Stem tokens
    tokens = [stemmer.stem(token) for token in tokens]
    return tokens

# Build a vocabulary from the raw descriptions (lowercased tokens)
raw_vocab = set()
for text in descriptions:
    tokens = word_tokenize(text.lower())
    raw_tokens = [token for token in tokens if token not in string.punctuation]
    raw_vocab.update(raw_tokens)

# Build a vocabulary from the preprocessed descriptions
processed_vocab = set()
for text in descriptions:
    tokens = preprocess_text(text)
    processed_vocab.update(tokens)

print("Vocabulary size before preprocessing:", len(raw_vocab))
print("Vocabulary size after preprocessing:", len(processed_vocab))

# Create a new column 'preprocessed_description' by applying the preprocessing␣
 ↪function
df_meta['preprocessed_description'] = df_meta['description'].apply(lambda x: '␣
 ↪'.join(preprocess_text(x)))

# Print a sample original and its preprocessed description
sample_index = 0  # you can change the index to view a different sample
print("Original Description:")
print(df_meta['description'].iloc[sample_index])
print("\nPreprocessed Description:")
print(df_meta['preprocessed_description'].iloc[sample_index])
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\david\AppData\Roaming\nltk_data…
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\david\AppData\Roaming\nltk_data…
[nltk_data]   Package stopwords is already up-to-date!

Vocabulary size before preprocessing: 51296
Vocabulary size after preprocessing: 27243
Original Description:
```

```
BEHRINGER EUROPOWER EPQ900. Professional 900-Watt Light Weight Stereo Power
Amplifier with ATR (Accelerated Transient Response) Technology. 2 x 390 Watts
into 4 Ohms; 2 x 245 Watts into 8 Ohms. 2 x 390 Watts into 4 Ohms; 2 x 245 Watts
into 8 Ohms. ATR (Accelerated Transient Response) technology for ultimate punch
and clarity. ATR (Accelerated Transient Response) technology for ultimate punch
and clarity. Ultra-light, ultra-low noise and ultra-efficient switch-mode power
supply for noise-free audio, superior transient response and low power
consumption. Ultra-light, ultra-low noise and ultra-efficient switch-mode power
supply for noise-free audio, superior transient response and low power
consumption. Independent limiters for each channel offer maximum output level
with reliable overload protection. Independent limiters for each channel offer
maximum output level with reliable overload protection. Detented gain controls
for precise setting and matching of sensitivity. Detented gain

Preprocessed Description:
behring europow profession light weight stereo power amplifi atr acceler
transient respons technolog x watt ohm x watt ohm x watt ohm x watt ohm atr
acceler transient respons technolog ultim punch clariti atr acceler transient
respons technolog ultim punch clariti ultra-light ultra-low nois ultra-effici
switch-mod power suppli noise-fre audio superior transient respons low power
consumpt ultra-light ultra-low nois ultra-effici switch-mod power suppli noise-
fre audio superior transient respons low power consumpt independ limit channel
offer maximum output level reliabl overload protect independ limit channel offer
maximum output level reliabl overload protect detent gain control precis set
match sensit detent gain
```

3. Represent each item in the TF-IDF vector space. You can use your preferred library for text representation, for example, scikit-learn or gensim.

[130]:
```
from sklearn.feature_extraction.text import TfidfVectorizer

# Use preprocessed text and split on spaces without additional preprocessing.
tfidf_vectorizer = TfidfVectorizer(tokenizer=lambda x: x.split(),␣
 ↪lowercase=False)
tfidf_matrix = tfidf_vectorizer.
 ↪fit_transform(df_meta['preprocessed_description'])

print("TF-IDF matrix shape:", tfidf_matrix.shape)
```

```
c:\Users\david\anaconda3\envs\WRS\lib\site-
packages\sklearn\feature_extraction\text.py:517: UserWarning: The parameter
'token_pattern' will not be used since 'tokenizer' is not None'
  warnings.warn(

TF-IDF matrix shape: (23984, 27243)
```

4. Represent each item using pretrained word embeddings (e.g., GloVe, word2vec).

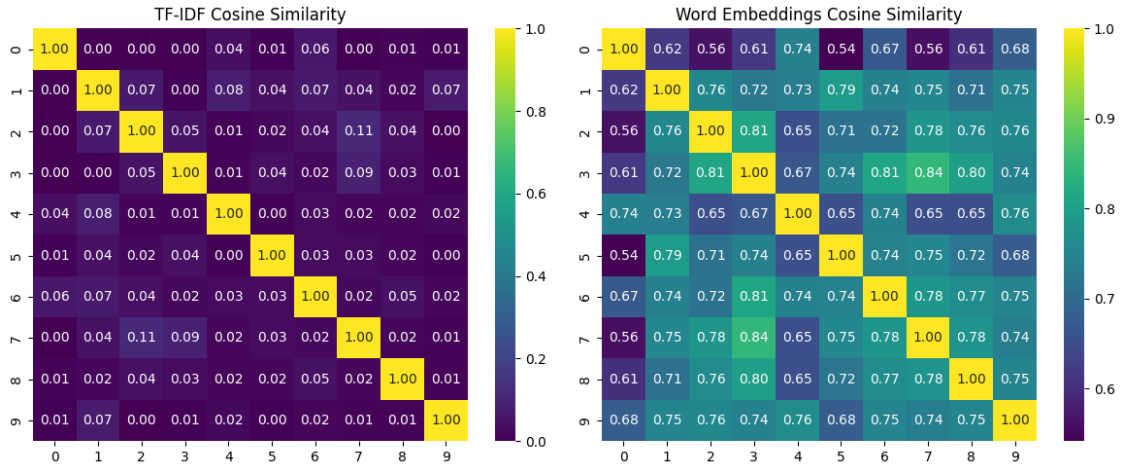
[131]:
```
import os
import gensim
import gensim.downloader as api
import numpy as np

w2v_path = 'Data/GoogleNews-vectors-negative300.bin'

try:
    word2vec = gensim.models.KeyedVectors.load_word2vec_format(w2v_path,␣
 ↪binary=True)
except FileNotFoundError:
    print("Downloading word2vec-google-news-300...")
    word2vec = api.load("word2vec-google-news-300")

def get_avg_word2vec(tokens, model):
    vectors = [model[token] for token in tokens if token in model]
    return np.mean(vectors, axis=0) if vectors else np.zeros(model.vector_size)

df_meta['w2v_vector'] = df_meta['preprocessed_description'].apply(lambda txt:␣
 ↪get_avg_word2vec(txt.split(), word2vec))

print("Sample word2vec representation (vector shape):", df_meta['w2v_vector'].
 ↪iloc[0].shape)
```

```
Downloading word2vec-google-news-300…
Sample word2vec representation (vector shape): (300,)
```

5. Explore the similarity between items within the vector spaces by computing their cosine similarity. Compare results obtained with TF-IDF and the word embeddings. Discuss what you find. Note that you can select a subset of items to better highlight the differences between the two text representations.

[132]:
```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Select a subset of items (e.g., the first 10 items with more than 900␣
 ↪characters)
subset = df_meta[df_meta['desc_len'] > 900].iloc[:10]

# TF-IDF similarity
subset_tfidf = tfidf_matrix[subset.index, :]
tfidf_sim = cosine_similarity(subset_tfidf)

# Word Embedding similarity: stack the w2v_vector representations into a matrix
subset_w2v = np.stack(subset['w2v_vector'].values)
w2v_sim = cosine_similarity(subset_w2v)
```

```
# Plot similarity matrices side by side
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
sns.heatmap(tfidf_sim, annot=True, fmt=".2f", cmap="viridis")
plt.title("TF-IDF Cosine Similarity")

plt.subplot(1, 2, 2)
sns.heatmap(w2v_sim, annot=True, fmt=".2f", cmap="viridis")
plt.title("Word Embeddings Cosine Similarity")

plt.tight_layout()
plt.show()
```

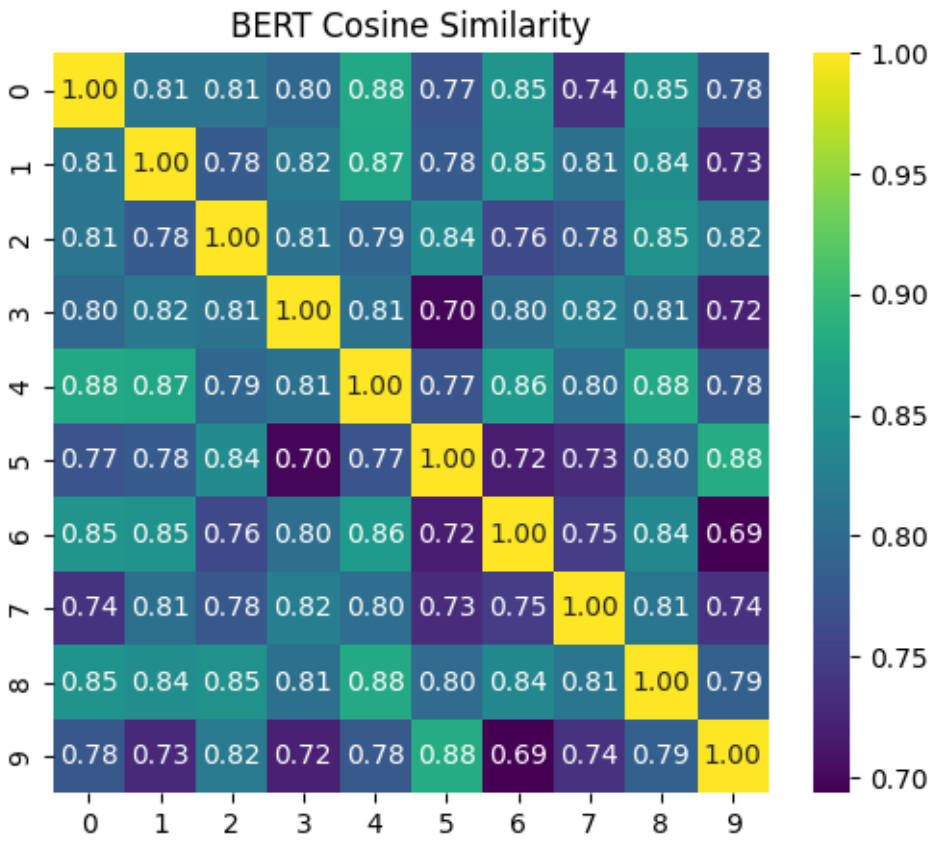


6. [Optional] Represent each item rated by the users using the word vectors from the last layer of BERT. Here, you can directly use the vectors from the pretrained version of BERT available in huggingface. To load the model, install the Transformers library and PyTorch.

[133]:
```
import torch
from transformers import BertModel, BertTokenizer

!pip install transformers torch --quiet


# Load model and tokenizer
model_name = "bert-base-uncased"
tokenizer = BertTokenizer.from_pretrained(model_name)
model = BertModel.from_pretrained(model_name)
model.eval()  # set model to evaluation mode
```

```
# Optionally, move model to GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

# Function to get BERT representation (using the [CLS] token) for a given text.
def get_bert_vector(text):
    # Handle potential issues with NaN or empty strings
    if not text or text.lower() == "nan":
        return torch.zeros(model.config.hidden_size)
    # Tokenize text with truncation to max_length 512 tokens.
    inputs = tokenizer(text, return_tensors="pt", truncation=True,␣
 ↪max_length=512, padding="max_length")
    inputs = {k: v.to(device) for k, v in inputs.items()}
    with torch.no_grad():
        outputs = model(**inputs)
    # Use [CLS] token (first token) from the last hidden state as the␣
 ↪representation.
    cls_vector = outputs.last_hidden_state[:, 0, :].squeeze().cpu()
    return cls_vector

# Compute BERT representations for items in the 'description' column of the␣
 ↪subset.
bert_vectors = subset['description'].apply(lambda x: get_bert_vector(str(x)))
print("BERT representations computed for each item.")

# Convert BERT tensors to numpy arrays and stack them
bert_matrix = np.stack(bert_vectors.apply(lambda vec: vec.numpy()).tolist())
bert_sim = cosine_similarity(bert_matrix)

# Plot the cosine similarity matrix for BERT representations
plt.figure(figsize=(6, 5))
sns.heatmap(bert_sim, annot=True, fmt=".2f", cmap="viridis")
plt.title("BERT Cosine Similarity")
plt.show()
```

```
BERT representations computed for each item.
```



0.0.4 Week 10 [Mar 3 - Mar 9]: Content-Based Recommender System

In this week, we will continue using the train and test splits defined in Week 6. - Note: similar to the task in Week 9, for this part, you are free to only use a subset of the metadata file, e.g., only using items that are in the train/test splits or using the first n characters/words of the text content. You are also welcome to use additional metadata (e.g., by crawling the internet). Please report and justify your choice.

[134]:
```
print("Number of rows:", df_meta.shape[0])
print("Number of fields:", df_meta.shape[1])
max_length = df_meta['description'].astype(str).apply(len).max()
print("Max length of strings in 'description':", max_length)
print("\nColumns and their data types:")
print(df_meta.dtypes)
```

```
Number of rows: 23984
Number of fields: 6
Max length of strings in 'description': 1000

Columns and their data types:
item_id                    object
```

```
title                          object
description                    object
desc_len                        int64
preprocessed_description       object
w2v_vector                     object
dtype: object
```

- Transform the description column of each item into a TF-IDF representation or other numerical value, e.g., token-count based, that can represent the summaries. Select at least one other factor that can be used as an item feature, for example title. Apply the appropriate preprocessing on the features. These choices should also be reported and justified.

[135]:
```
from sklearn.feature_extraction.text import TfidfVectorizer
from scipy.sparse import hstack
import numpy as np
import random
from tqdm import tqdm
from sklearn.metrics.pairwise import cosine_similarity

# Preprocess the title column using the same preprocessing function already␣
 ↪defined for descriptions.
# (Assuming there is a 'title' column in df_meta)
df_meta['preprocessed_title'] = df_meta['title'].astype(str).apply(lambda x: '␣
 ↪'.join(preprocess_text(x)))

# Build a TF-IDF representation for the preprocessed titles.
title_tfidf_vectorizer = TfidfVectorizer(tokenizer=lambda x: x.split(),␣
 ↪lowercase=False)
tfidf_title_matrix = title_tfidf_vectorizer.
 ↪fit_transform(df_meta['preprocessed_title'])

# Combine the description TF-IDF matrix (tfidf_matrix, already computed) with␣
 ↪the title TF-IDF matrix.
combined_features = hstack([tfidf_matrix, tfidf_title_matrix])

# For each item, store its combined feature vector as a new column.
# Each row is a sparse vector; we extract it as a CSR matrix slice.
df_meta['combined_features'] = [combined_features.getrow(i) for i in␣
 ↪range(combined_features.shape[0])]

print("TF-IDF description vector length:", tfidf_matrix.shape[1])
print("TF-IDF title vector length:", tfidf_title_matrix.shape[1])
print("Combined feature vector length:", combined_features.shape[1])

# Sample 100 random items from the metadata
random.seed(42)  # For reproducibility
sample_indices = random.sample(range(len(df_meta)), 200)
```

```
sample_items = df_meta.iloc[sample_indices].copy()

# Create a progress bar for the similarity computation
print("Computing pairwise similarities...")
similarities = np.zeros((200, 200))
item_pairs = []

for i in tqdm(range(200)):
    for j in range(i+1, 200):
        # Extract the TF-IDF vectors for the two items
        item_i_vec = sample_items['combined_features'].iloc[i]
        item_j_vec = sample_items['combined_features'].iloc[j]

        # Calculate cosine similarity
        sim = cosine_similarity(item_i_vec, item_j_vec)[0][0]
        similarities[i, j] = sim
        similarities[j, i] = sim  # Symmetric matrix

        # Store the item pair and their similarity
        item_pairs.append((i, j, sim))

# Find the most similar pair (highest cosine similarity)
most_similar = max(item_pairs, key=lambda x: x[2])
i_most, j_most, sim_most = most_similar

print(f"\nMost similar items (similarity = {sim_most:.4f}):")
print("\nItem 1:")
print(f"Title: {sample_items['title'].iloc[i_most]}")
print(f"Description excerpt: {sample_items['description'].iloc[i_most][:300]}...
 ↪")

print("\nItem 2:")
print(f"Title: {sample_items['title'].iloc[j_most]}")
print(f"Description excerpt: {sample_items['description'].iloc[j_most][:300]}...
 ↪")

# Find the least similar pair (lowest cosine similarity)
least_similar = min(item_pairs, key=lambda x: x[2])
i_least, j_least, sim_least = least_similar

print(f"\nLeast similar items (similarity = {sim_least:.4f}):")
print("\nItem 1:")
print(f"Title: {sample_items['title'].iloc[i_least]}")
print(f"Description excerpt: {sample_items['description'].iloc[i_least][:300]}..
 ↪.")

print("\nItem 2:")
```

```
print(f"Title: {sample_items['title'].iloc[j_least]}")
print(f"Description excerpt: {sample_items['description'].iloc[j_least][:300]}..
  ↪.")
```

```
c:\Users\david\anaconda3\envs\WRS\lib\site-
packages\sklearn\feature_extraction\text.py:517: UserWarning: The parameter
'token_pattern' will not be used since 'tokenizer' is not None'
  warnings.warn(

TF-IDF description vector length: 27243
TF-IDF title vector length: 14307
Combined feature vector length: 41550
Computing pairwise similarities…

100%|      | 200/200 [00:15<00:00, 12.70it/s]


Most similar items (similarity = 0.9796):

Item 1:
Title: Guitar Chord & Fretboard Note Chart Instructional Easy 11"x17" Poster for
Beginners Chords & Notes | A New Song Music |
Description excerpt: nan…

Item 2:
Title: A New Song Music Laminated Guitar Chord & Fretboard Note Chart
Instructional Easy Poster for Beginners Chords & Notes 11"x17"
Description excerpt: nan…

Least similar items (similarity = 0.0000):

Item 1:
Title: Guitar Roller String Trees String Tree Guide Retainer Replacement Guitar
Parts for Electric Guitar Accessory Set (Chrome .10Pcs)
Description excerpt: Features:. 100% Brand new and high quality The rollers
lessen the stress on your strings which allows the guitar to stay in tune
longer. This also prevents wear and tear on the strings which results in longer
string life. Base is about 6mm (1/4") wide x 11.5mm (7/16") long Total height
including rol…

Item 2:
Title: Hosa HXS-005 REAN XLR3F to 1/4" TRS Pro Balanced Interconnect Cable, 5
Feet
Description excerpt: The Hosa HXS005 Rean Pro 1/4 Inch TRS to XLR3 Female Cable
is designed to connect gear with XLR outputs to gear with balanced phone inputs.
It is ideal for use in touring and other live-sound applications…
```

- After you represent each item in a vector space, represent each user in the same vector space.

This can be done by using a simple average of the items the user rated. Note that you can also try to implement better ways to build the user representation.

[136]:
```
# Build a mapping from item_id (from metadata) to its combined feature vector␣
↪(a sparse row)
item_profiles = dict(zip(df_meta['item_id'], df_meta['combined_features']))

# Determine the number of features from the first item's vector
num_features = df_meta['combined_features'].iloc[0].toarray().shape[1]

# Create user profiles using only reviews in the training set with ratings >= 1.
user_profiles = {}
for inner_uid, ratings in trainset.ur.items():
    raw_uid = trainset.to_raw_uid(inner_uid)
    # Collect the dense vectors of items rated >= 1 by the user.
    vectors = []
    for inner_iid, rating in ratings:
        if rating >= 1:
            raw_iid = trainset.to_raw_iid(inner_iid)
            if raw_iid in item_profiles:
                # Convert the sparse item vector to a dense array.
                item_vector = item_profiles[raw_iid].toarray().flatten()
                vectors.append(item_vector)
    # Compute the average of the selected vectors, or initialize with a zero␣
↪vector if none.
    if vectors:
        user_profile = np.mean(vectors, axis=0)
    else:
        user_profile = np.zeros(num_features)
    user_profiles[raw_uid] = user_profile

print("Created user profiles for {} users using only items with ratings >= 1.".
↪format(len(user_profiles)))
```

```
Created user profiles for 800 users using only items with ratings >= 1.
```

- Calculate the user-item rating prediction for an item by using a similarity/distance metric between the user and the item representations. A metric such as cosine similarity or Euclidean distance can be used. Motivate your choice.

[137]:
```
import random
from sklearn.metrics import mean_squared_error

def predict_rating(user_id, item_id, min_rating=1, max_rating=5):
    # Retrieve the user profile vector and item feature vector.
    user_vector = user_profiles.get(user_id)
    if user_vector is None:
        print("Unknown user.")
```

```
        return None
    if item_id not in item_profiles:
        print("Unknown item.")
        return None

    # Convert the sparse item vector to a dense representation.
    item_vector = item_profiles[item_id].toarray().flatten()

    # Do NOT mean-center the item vector, since our user profiles are built␣
↪from raw item vectors.
    # Reshape vectors to 2D arrays for cosine_similarity.
    user_vector_2d = user_vector.reshape(1, -1)
    item_vector_2d = item_vector.reshape(1, -1)

    # Compute cosine similarity between the user and item vectors.
    similarity = cosine_similarity(user_vector_2d, item_vector_2d)[0][0]

    # If your cosine similarities are in [0, 1] (common with non-negative␣
↪vectors),
    # you may want to map directly as follows:
    predicted_rating = similarity * (max_rating - min_rating) + min_rating
    #
    # Otherwise, if similarities can span [-1, 1], you can use:
    # predicted_rating = (similarity + 1) * (max_rating - min_rating) / 2 +␣
↪min_rating
    return predicted_rating

# Sample 100 random (user_id, item_id, rating) tuples from the test set.
sample_df = test_df.sample(n=100, random_state=42)

true_ratings = []
predicted_ratings_model = []    # Using predict_rating (content-based)
predicted_ratings_random = []   # Random predictor (uniformly random 1 to 5)
predicted_ratings_constant = [] # Constant predictor (always predict 3)

for _, row in sample_df.iterrows():
    user_id = row['user_id']
    item_id = row['item_id']
    true_rating = row['rating']

    # Content-based predictor from our model.
    pred = predict_rating(user_id, item_id)
    if pred is not None:
        true_ratings.append(true_rating)
        predicted_ratings_model.append(pred)
    else:
        print(f"Prediction not available for user {user_id} and item {item_id}")
```

```
    # Random predictor: generate a random float rating between 1 and 5.
    random_rating = random.uniform(1, 5)
    predicted_ratings_random.append(random_rating)

    # Constant predictor: always predict 3.
    predicted_ratings_constant.append(3)

# Compute RMSE for each predictor.
mse_model = mean_squared_error(true_ratings, predicted_ratings_model)
rmse_model = mse_model ** 0.5

mse_random = mean_squared_error(true_ratings, predicted_ratings_random)
rmse_random = mse_random ** 0.5

mse_constant = mean_squared_error(true_ratings, predicted_ratings_constant)
rmse_constant = mse_constant ** 0.5

print("RMSE for content-based predictor:", rmse_model)
print("RMSE for random predictor (ratings between 1-5):", rmse_random)
print("RMSE for constant predictor (always predict 3):", rmse_constant)
```

```
RMSE for content-based predictor: 3.1830293882582072
RMSE for random predictor (ratings between 1-5): 2.178067695909589
RMSE for constant predictor (always predict 3): 1.7378147196982767
```

- Report Precision@10, MAP@10, MRR@10, hit rate and coverage using ratings 4 in the test set. Compare the results with the models from previous weeks.

[138]:
```
from numpy.linalg import norm
import numpy as np
from tqdm import tqdm
import random

# Precompute each item's raw vector and its norm (without subtracting the␣
 ↪global mean).
item_vectors = {}
item_norms = {}
for item in item_profiles:
    vec = item_profiles[item].toarray().flatten()  # use raw vector
    item_vectors[item] = vec
    item_norms[item] = norm(vec)

# Use a random sample of 100 users (or choose your desired subset)
# sample_users = random.sample(list(user_rating_counts.keys()), 10)
sample_users = list(user_profiles.keys())
recs_sample = {}
all_items = set(item_profiles.keys())
```

```
# Process each user.
for user in tqdm(sample_users, desc="Processing sample_df users"):
    try:
        inner_uid = trainset.to_inner_uid(user)
        rated_items = {trainset.to_raw_iid(inner_iid) for inner_iid, _ in␣
 ↪trainset.ur[inner_uid]}
    except Exception as e:
        rated_items = set()  # Assume no rated items if user not in training␣
 ↪set.

    unobserved_items = list(all_items - rated_items)
    predictions = []
    user_vec = user_profiles[user]
    user_norm = norm(user_vec)

    for item in unobserved_items:
        vec = item_vectors[item]
        item_n = item_norms[item]
        if user_norm == 0 or item_n == 0:
            sim = 0
        else:
            sim = np.dot(user_vec, vec) / (user_norm * item_n)
        predictions.append((item, sim))

    # Sort predictions by descending similarity and save top 25
    predictions.sort(key=lambda x: x[1], reverse=True)
    top_25 = predictions[:25]
    recs_sample[user] = top_25

# For evaluation, use only the top 10 items from the top 25 list.
recs_sample_eval = {user: [item for item, sim in recs_sample[user][:10]] for␣
 ↪user in recs_sample}

# Filter ground_truth to only include users in sample_users.
filtered_ground_truth = {user: ground_truth[user] for user in sample_users if␣
 ↪user in ground_truth}

# Evaluate the recommendations using your provided evaluate function.
results = evaluate(recs_sample_eval, filtered_ground_truth, k=10,␣
 ↪total_items=len(item_profiles))
print("Evaluation metrics for sample_df users (Hit Rate, Precision@10, MAP@10,␣
 ↪MRR@10, Coverage):", results)

# Print for each user the 10 recommended items and their cosine similarity␣
 ↪scores.
```

```
#print("\nRecommended items with cosine similarity for each user (top 25␣
 ↪computed, top 10 evaluated):")
#for user in recs_sample:
#    print(f"\nUser: {user}")
#    for item, sim in recs_sample[user][:10]:
#        print(f"   {item}: cosine similarity = {sim:.4f}")
```

```
Processing sample_df users: 100%|      | 800/800 [43:39<00:00,  3.27s/it]
```

```
Evaluation metrics for sample_df users (Hit Rate, Precision@10, MAP@10, MRR@10,
Coverage): (0.012135922330097087, 0.0012135922330097086, 0.0007258733715044396,
0.0022451456310679614, 0.04853235490326885)
```

0.0.5 6 Week 11 [Mar 10 - Mar 16] Hybrid Recommender System

Create three hybrid recommender systems by combining a collaborative filtering model with a content-based model using the following three strategies: - Parallel combination strategy that re-ranks the items by combining the individual rankings from the two models with some aggregation function such as the sum, average, minimum or maximum

[139]:
```
def create_hybrid_parallel(cf_recs, cb_recs, users, k=10, aggregation='sum'):
    """
    Create hybrid recommendations by combining collaborative filtering and␣
 ↪content-based rankings.

    Parameters:
    cf_recs (dict): Dictionary mapping users to their collaborative filtering␣
 ↪recommendations
    cb_recs (dict): Dictionary mapping users to their content-based␣
 ↪recommendations
    users (list): List of users to generate recommendations for
    k (int): Number of recommendations to generate
    aggregation (str): Aggregation function ('sum', 'avg', 'min', 'max')

    Returns:
    dict: Dictionary mapping users to their hybrid recommendations
    """
    hybrid_recs = {}

    for user in users:
        # Get recommendations from both models
        cf_items = cf_recs.get(user, [])
        cb_items = cb_recs.get(user, [])

        # Skip if both recommendation lists are empty
        if not cf_items and not cb_items:
            continue
```

```
        # Create a dictionary to store combined scores
        item_scores = {}

        # Assign scores based on ranking in collaborative filtering␣
 ↪recommendations
        for i, item in enumerate(cf_items):
            # Higher score for higher ranked items (inverse of position)
            item_scores[item] = k - i if i < k else 0

        # Combine with scores from content-based recommendations
        for i, item in enumerate(cb_items):
            score = k - i if i < k else 0
            if item in item_scores:
                if aggregation == 'sum':
                    item_scores[item] += score
                elif aggregation == 'avg':
                    item_scores[item] = (item_scores[item] + score) / 2
                elif aggregation == 'min':
                    item_scores[item] = min(item_scores[item], score)
                elif aggregation == 'max':
                    item_scores[item] = max(item_scores[item], score)
            else:
                item_scores[item] = score

        # Sort items by combined score in descending order
        sorted_items = sorted(item_scores.items(), key=lambda x: x[1],␣
 ↪reverse=True)

        # Select top-k items
        hybrid_recs[user] = [item for item, _ in sorted_items[:k]]

    return hybrid_recs

# Test different aggregation methods for the hybrid recommender
aggregation_methods = ['sum']

for method in aggregation_methods:
    # Create hybrid recommendations
    hybrid_recs = create_hybrid_parallel(knn_recs, recs_sample, sample_users,␣
 ↪aggregation=method)

    # Evaluate the hybrid recommendations
    parallel_results = evaluate(hybrid_recs, filtered_ground_truth, k=10,␣
 ↪total_items=len(item_profiles))
    print(f"Hybrid Recommender ({method.capitalize()} Aggregation):",␣
 ↪parallel_results)
```

```
# Compare with individual recommenders
print("\nComparison with individual recommenders:")
print("Content-Based:", results)  # Results from previous cell
print("KNN Collaborative Filtering:", knn_metrics)  # From previous evaluations
```

```
Hybrid Recommender (Sum Aggregation): (0.05825242718446602,
0.005825242718446603, 0.004802267347117494, 0.018616119586993374,
0.0943545697131421)

Comparison with individual recommenders:
Content-Based: (0.012135922330097087, 0.0012135922330097086,
0.0007258733715044396, 0.0022451456310679614, 0.04853235490326885)
KNN Collaborative Filtering: (0.10922330097087378, 0.011650485436893206,
0.00841981197641613, 0.03142144398212359, 0.587426326129666)
```

- Switching strategy that uses the recommendations from the collaborative filtering model for some users and the recommendations from the content-based model for other users chosen by a predefined condition.

[140]:
```
def create_hybrid_switching(cf_recs, cb_recs, user_rating_counts, users,␣
↪rating_threshold=None):
    """
    Create hybrid recommendations using a switching strategy based on user␣
↪activity.

    Parameters:
    cf_recs (dict): Dictionary mapping users to their collaborative filtering␣
↪recommendations
    cb_recs (dict): Dictionary mapping users to their content-based␣
↪recommendations
    user_rating_counts (dict): Dictionary mapping users to their number of␣
↪ratings
    users (list): List of users to generate recommendations for
    rating_threshold (int): Threshold for switching between models (if None,␣
↪use median)

    Returns:
    dict: Dictionary mapping users to their hybrid recommendations
    tuple: Statistics about the switching (count of CF users, count of CB users)
    """
    hybrid_recs = {}

    # If no threshold provided, use the median number of ratings
    if rating_threshold is None:
        ratings = [user_rating_counts.get(user, 0) for user in users]
        rating_threshold = np.median(ratings)
```

```
    cf_user_count = 0
    cb_user_count = 0

    for user in users:
        # Determine which model to use based on user activity
        rating_count = user_rating_counts.get(user, 0)

        if rating_count >= rating_threshold:
            # Active users get collaborative filtering recommendations
            if user in cf_recs:
                hybrid_recs[user] = cf_recs[user]
                cf_user_count += 1
        else:
            # Less active users get content-based recommendations
            if user in cb_recs:
                hybrid_recs[user] = cb_recs[user]
                cb_user_count += 1

    return hybrid_recs, (cf_user_count, cb_user_count)

# Try different thresholds for switching
thresholds = [5]  # None will use median

for threshold in thresholds:
    # Create hybrid recommendations using switching strategy
    hybrid_recs, stats = create_hybrid_switching(
        knn_recs, recs_sample, user_rating_counts, sample_users,␣
 ↪rating_threshold=threshold
    )

    # Evaluate the hybrid recommendations
    switching_results = evaluate(hybrid_recs, filtered_ground_truth, k=10,␣
 ↪total_items=len(item_profiles))

    threshold_desc = f"median ({stats[0]+stats[1]//2})" if threshold is None␣
 ↪else threshold
    print(f"Hybrid Switching Strategy (threshold={threshold_desc}):",␣
 ↪switching_results)
    print(f"  Users with CF recommendations: {stats[0]}, Users with CB␣
 ↪recommendations: {stats[1]}")

# Compare with individual recommenders
print("\nComparison with individual recommenders:")
print("Content-Based:", results)
print("KNN Collaborative Filtering:", knn_metrics)
```

```
Hybrid Switching Strategy (threshold=5): (0.0825242718446602,
```

```
0.008252427184466602, 0.00765537542112197, 0.0262145554014486,
0.0526184122748499)
  Users with CF recommendations: 761, Users with CB recommendations: 39

Comparison with individual recommenders:
Content-Based: (0.012135922330097087, 0.0012135922330097086,
0.0007258733715044396, 0.0022451456310679614, 0.04853235490326885)
KNN Collaborative Filtering: (0.10922330097087378, 0.011650485436893206,
0.00841981197641613, 0.03142144398212359, 0.587426326129666)
```

- Pipelining (sequential) strategy where a level of one model is used as input to the other model.

[141]:
```python
import pandas as pd
from surprise import Reader, Dataset, KNNWithMeans

# Assume you have already trained your CF model on synthetic ratings
# (e.g., using your build_cb_rating_df function on all sample users)
# and your model is stored in `knn_pipeline`.

def rerank_candidates_with_cf(precomputed_candidates, cf_model, top_k=10):
    """
    For each user, use the CF model to re-score the precomputed candidate items␣
 ↪and
    return the top_k items based on the CF prediction.
    precomputed_candidates is a dict mapping user -> list of (item, cb_sim)
    """
    final_recs = {}
    for user, candidates in precomputed_candidates.items():
        cf_scores = []
        for item, cb_sim in candidates:
            # Use CF model to predict the rating (or preference) for this␣
 ↪candidate item.
            pred = cf_model.predict(user, item)
            cf_scores.append((item, pred.est))
        # Sort by CF predicted score in descending order and select top_k
        cf_scores.sort(key=lambda x: x[1], reverse=True)
        final_recs[user] = [item for item, score in cf_scores[:top_k]]
    return final_recs

# Re-rank the precomputed top 25 candidates using the CF model.
final_recommendations = rerank_candidates_with_cf(recs_sample, knn_best,␣
 ↪top_k=10)

# (Optional) Evaluate the final recommendations.
# Filter ground_truth to only include users in sample_users.
filtered_ground_truth = {user: ground_truth[user] for user in sample_users if␣
 ↪user in ground_truth}
```

```
pipeline_results = evaluate(final_recommendations, filtered_ground_truth, k=10,␣
 ↪total_items=len(item_profiles))
print("Pipeline (CB candidate generation → CF re-ranking) final evaluation␣
 ↪metrics:")
print("Hit Rate, Precision@10, MAP@10, MRR@10, Coverage =", pipeline_results)

# Print final recommendations for each user.
# print("\nFinal CF re-ranked recommendations for each user:")
# for user in final_recommendations:
#     print(f"User {user}: {final_recommendations[user]}")
```

```
Pipeline (CB candidate generation → CF re-ranking) final evaluation metrics:
Hit Rate, Precision@10, MAP@10, MRR@10, Coverage = (0.019417475728155338,
0.0019417475728155341, 0.003059022961935583, 0.013349514563106794,
0.04861574382921948)
```

Use each hybrid model to rank the non-rated items for each user.

Regardless of the chosen option you need to Report Precision@10, MAP@10, MRR@10, the hit rate and coverage using ratings 4 in the test set. Compare the results with the models from previous weeks.

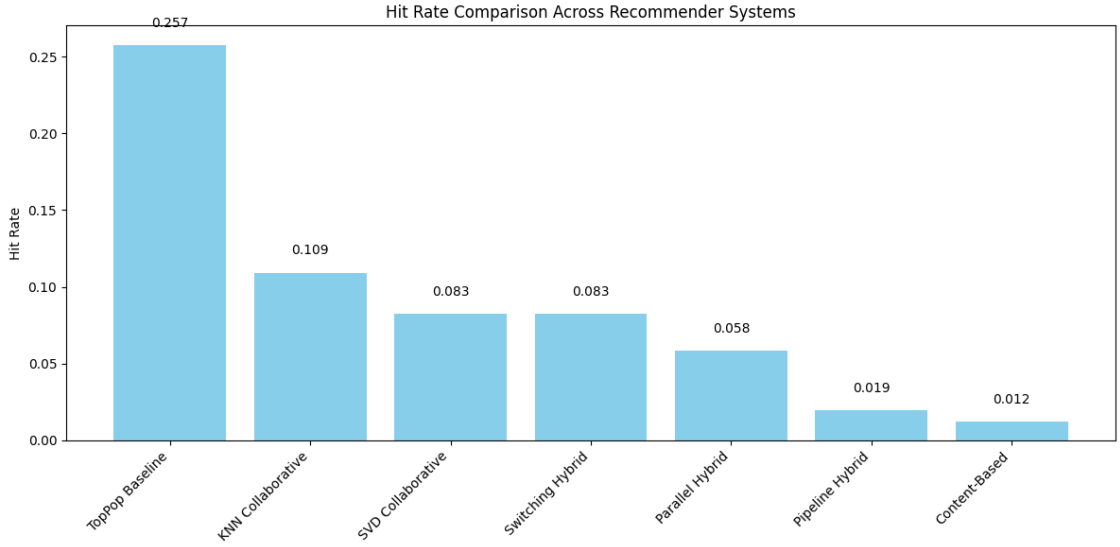
[142]:
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from matplotlib import cm

# Collect metrics from all recommender types
systems = {
    "Content-Based": results,
    "KNN Collaborative": knn_metrics,
    "SVD Collaborative": svd_metrics,
    "TopPop Baseline": top_pop_metrics
}

# Add best hybrid recommenders
# For Parallel hybrid
systems["Parallel Hybrid"] = parallel_results

# For Switching hybrid
systems["Switching Hybrid"] = switching_results

# For Pipeline hybrid, use the specific result with multiplier=2 instead of the␣
 ↪whole dictionary
systems["Pipeline Hybrid"] = pipeline_results

# Create a DataFrame to display the comparison
metrics_df = pd.DataFrame({
```

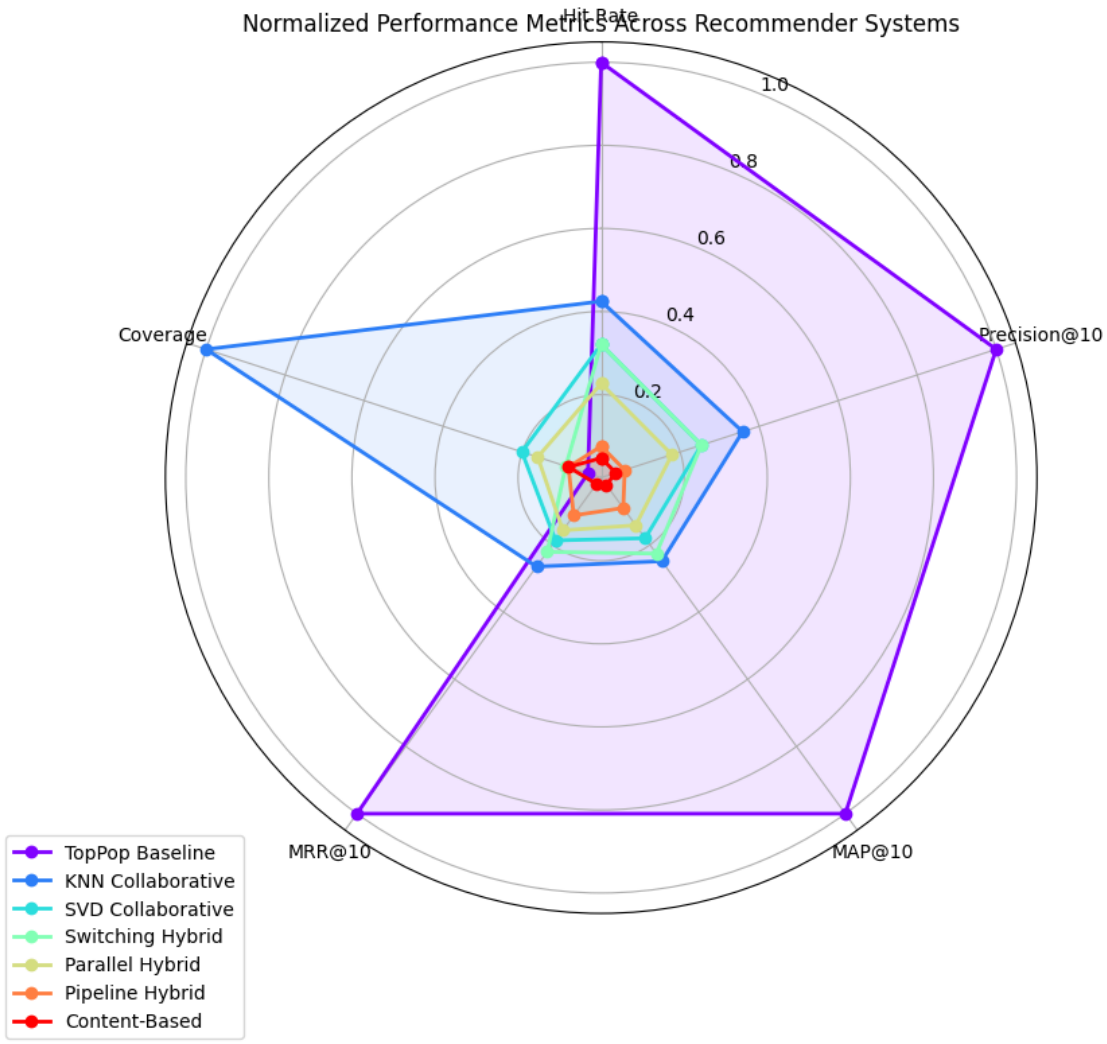
```
    "Recommender": list(systems.keys()),
    "Hit Rate": [systems[rec][0] for rec in systems],
    "Precision@10": [systems[rec][1] for rec in systems],
    "MAP@10": [systems[rec][2] for rec in systems],
    "MRR@10": [systems[rec][3] for rec in systems],
    "Coverage": [systems[rec][4] for rec in systems]
})

# Sort by Hit Rate descending
metrics_df = metrics_df.sort_values("Hit Rate", ascending=False)

# Display the table
print("Recommender System Performance Comparison:")
display(metrics_df)

# Create bar chart for visual comparison of hit rate
plt.figure(figsize=(12, 6))
bars = plt.bar(metrics_df["Recommender"], metrics_df["Hit Rate"],␣
 ↪color="skyblue")
plt.ylabel("Hit Rate")
plt.title("Hit Rate Comparison Across Recommender Systems")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()

# Add value labels on bars
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height + 0.01,
             f'{height:.3f}', ha='center', va='bottom')

plt.show()

# Create a radar chart to compare all metrics
metrics = ["Hit Rate", "Precision@10", "MAP@10", "MRR@10", "Coverage"]
# Normalize metrics for radar chart
normalized_df = metrics_df.copy()
for metric in metrics:
    max_val = normalized_df[metric].max()
    if max_val > 0:
        normalized_df[metric] = normalized_df[metric] / max_val

# Radar chart
colors = cm.rainbow(np.linspace(0, 1, len(systems)))

plt.figure(figsize=(10, 8))
angles = np.linspace(0, 2*np.pi, len(metrics), endpoint=False).tolist()
angles += angles[:1]  # Close the loop
```

```
ax = plt.subplot(111, polar=True)
ax.set_theta_offset(np.pi / 2)
ax.set_theta_direction(-1)

plt.xticks(angles[:-1], metrics)

for i, system in enumerate(normalized_df["Recommender"]):
    values = normalized_df.loc[normalized_df["Recommender"] == system, metrics].
 ↪values.flatten().tolist()
    values += values[:1]  # Close the loop
    ax.plot(angles, values, 'o-', linewidth=2, label=system, color=colors[i])
    ax.fill(angles, values, alpha=0.1, color=colors[i])

plt.legend(loc='upper right', bbox_to_anchor=(0.1, 0.1))
plt.title("Normalized Performance Metrics Across Recommender Systems")
plt.tight_layout()
plt.show()
```

Recommender System Performance Comparison:

	Recommender	Hit Rate	Precision@10	MAP@10	MRR@10	Coverage
3	TopPop Baseline	0.257282	0.032524	0.033864	0.118684	0.019646
1	KNN Collaborative	0.109223	0.011650	0.008420	0.031421	0.587426
2	SVD Collaborative	0.082524	0.008252	0.006104	0.022191	0.117878
5	Switching Hybrid	0.082524	0.008252	0.007655	0.026215	0.052618
4	Parallel Hybrid	0.058252	0.005825	0.004802	0.018616	0.094355
6	Pipeline Hybrid	0.019417	0.001942	0.003059	0.013350	0.048616
0	Content-Based	0.012136	0.001214	0.000726	0.002245	0.048532




Normalized Performance Metrics Across Recommender Systems
Hit Rate
Precision@10
MAP@10
MRR@10
Coverage
1.0
0.8
0.6
0.4
0.2
TopPop Baseline
KNN Collaborative
SVD Collaborative
Switching Hybrid
Parallel Hybrid
Pipeline Hybrid
Content-Based